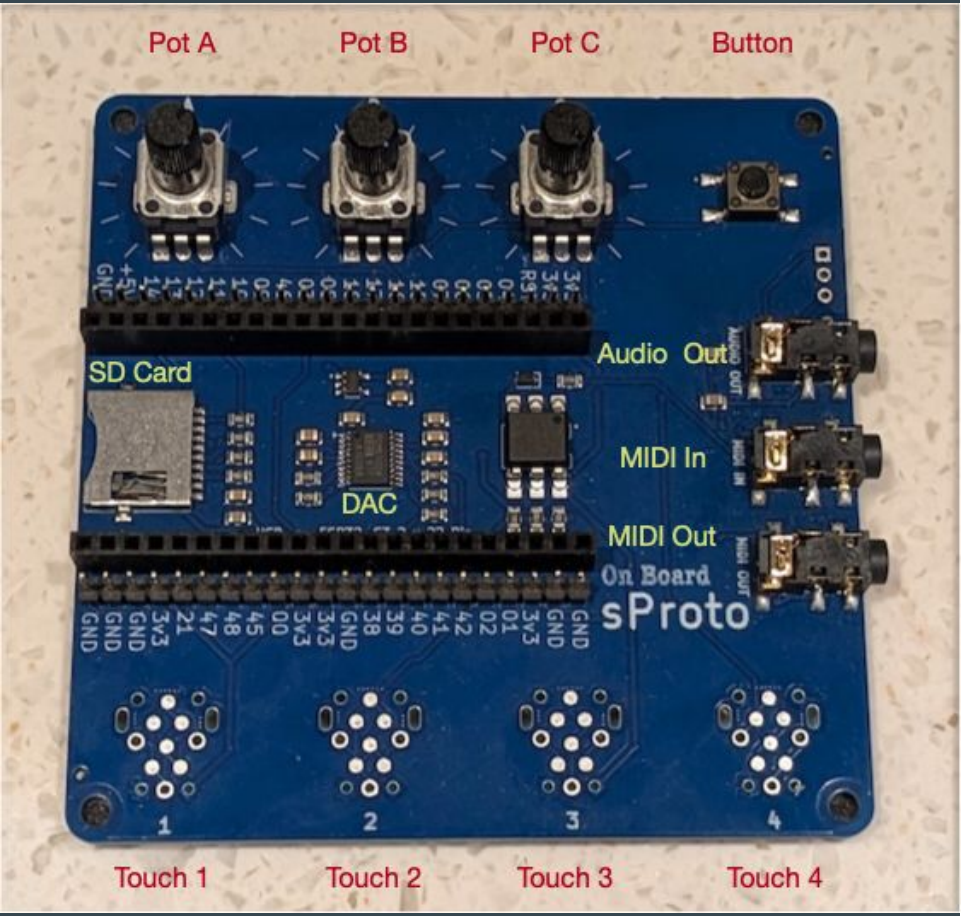
Coding a Digital Sound Box



On the ESP32 with the sProto PCB & Arduino IDE

Andrew R. Brown

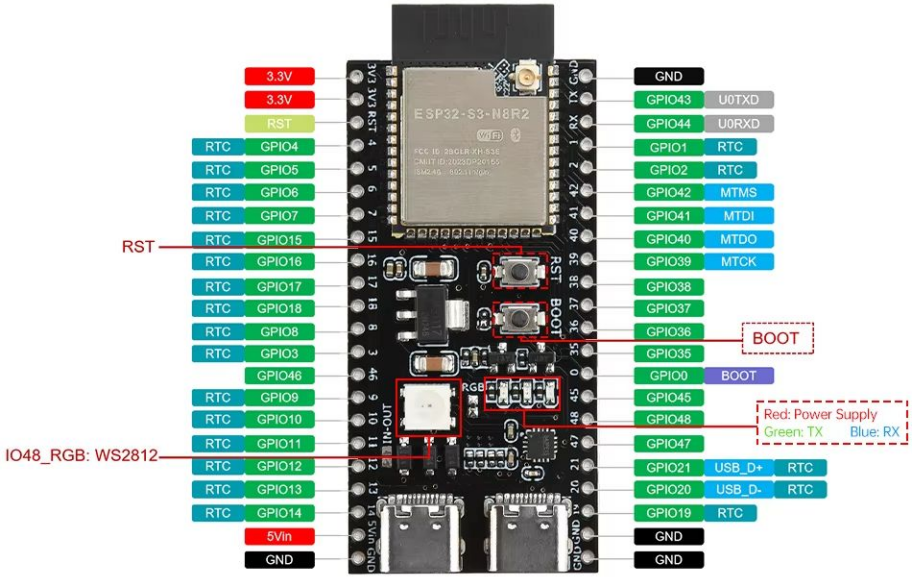


The sProto PCB

PCB for ESP32-S3 Development Board

Features:

- Pin Headers & Breakout Pins
- UI Controls (Pots, Button, Pads)
- I2S stereo audio output
- TRS A MIDI in and out
- Mini SD Card reader



ESP32-S3 Dev Board

- A dual-core XTensa LX7 MCU, running at 240 MHz.
- Integrated 2.4 GHz, 802.11 b/g/n Wi-Fi and Bluetooth 5 (LE)
- 512 KB of SRAM, 8 MB Flash, 16 MB PSRAM
- 2 x 22 pin headers
- “ESP32-S3-DevKitC-1” on Aliexpress or Ebay or Amazon


Playing with the
sProto Sound Box

sProto Sound Box

Four independent instruments on one sProto board:

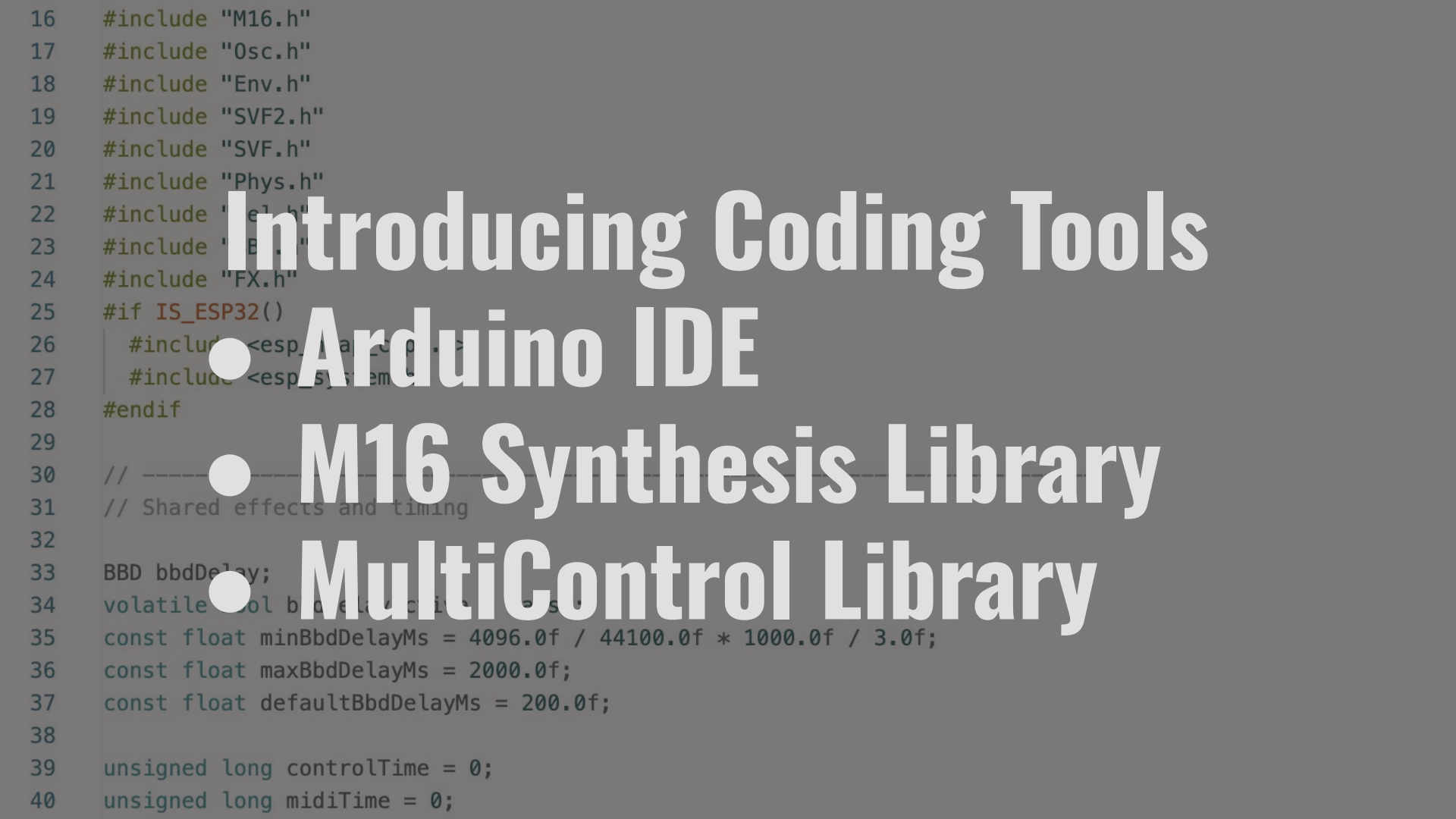
- Pad 1 / Dial 1: Tin Whistle, Trigger / Pitch
- Pad 2 / Dial 2: Kalimba, Trigger / Pitch
- Pad 3 / Dial 3: Spring, Trigger / Multiple Triggers
- Pad 4: Glockenspiel, Trigger

Hold the sProto button to adjust effects:

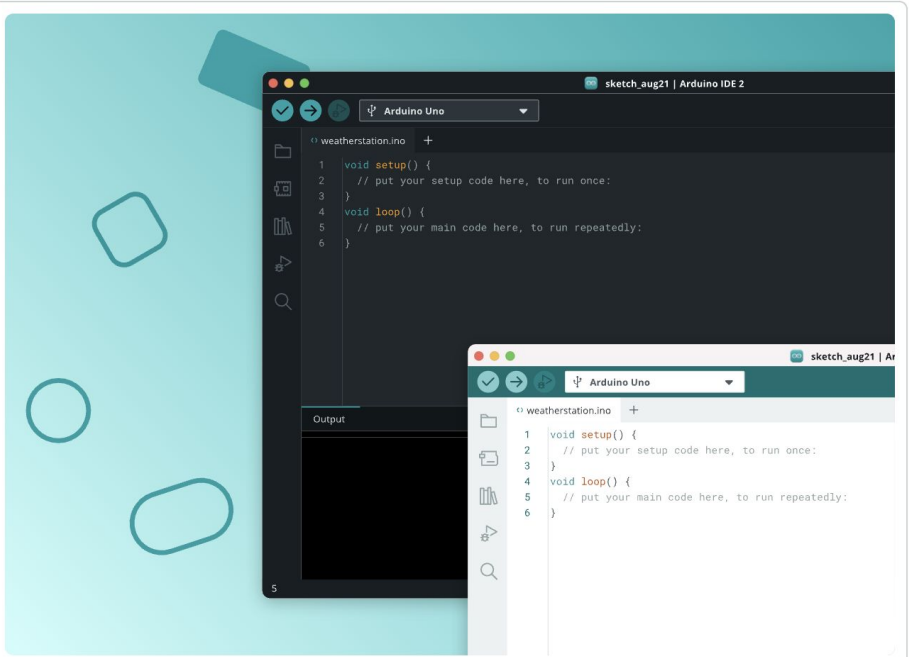
- Dial 1: Delay Time
- Dial 2: Delay Level
- Dial 3: Reverb Level

Introducing Coding Tools

- Arduino IDE
- M16 Synthesis Library
- MultiControl Library



Bring Your Projects to Life with Arduino Software



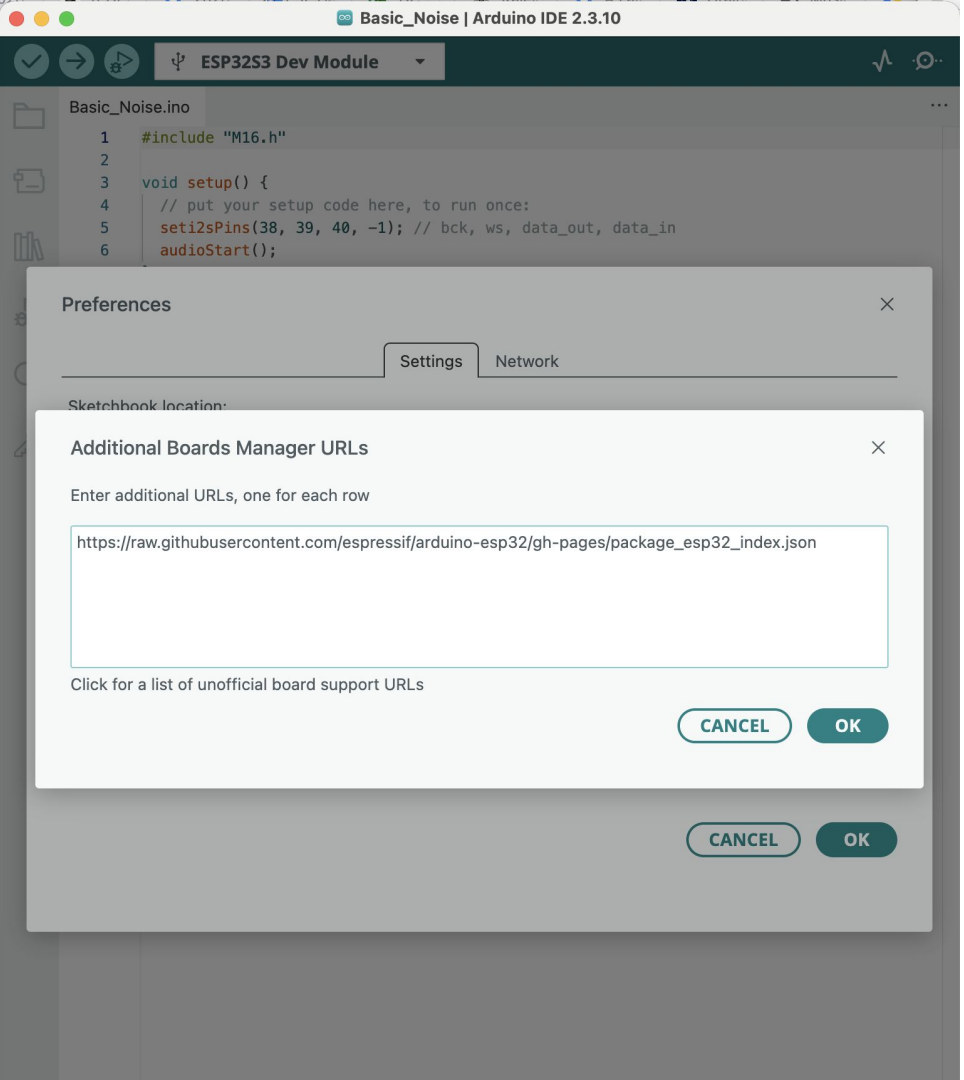
Arduino IDE 2.3.10

Release notes

Install the IDE

Integrated Development Environment

https://www.arduino.cc/en/software/

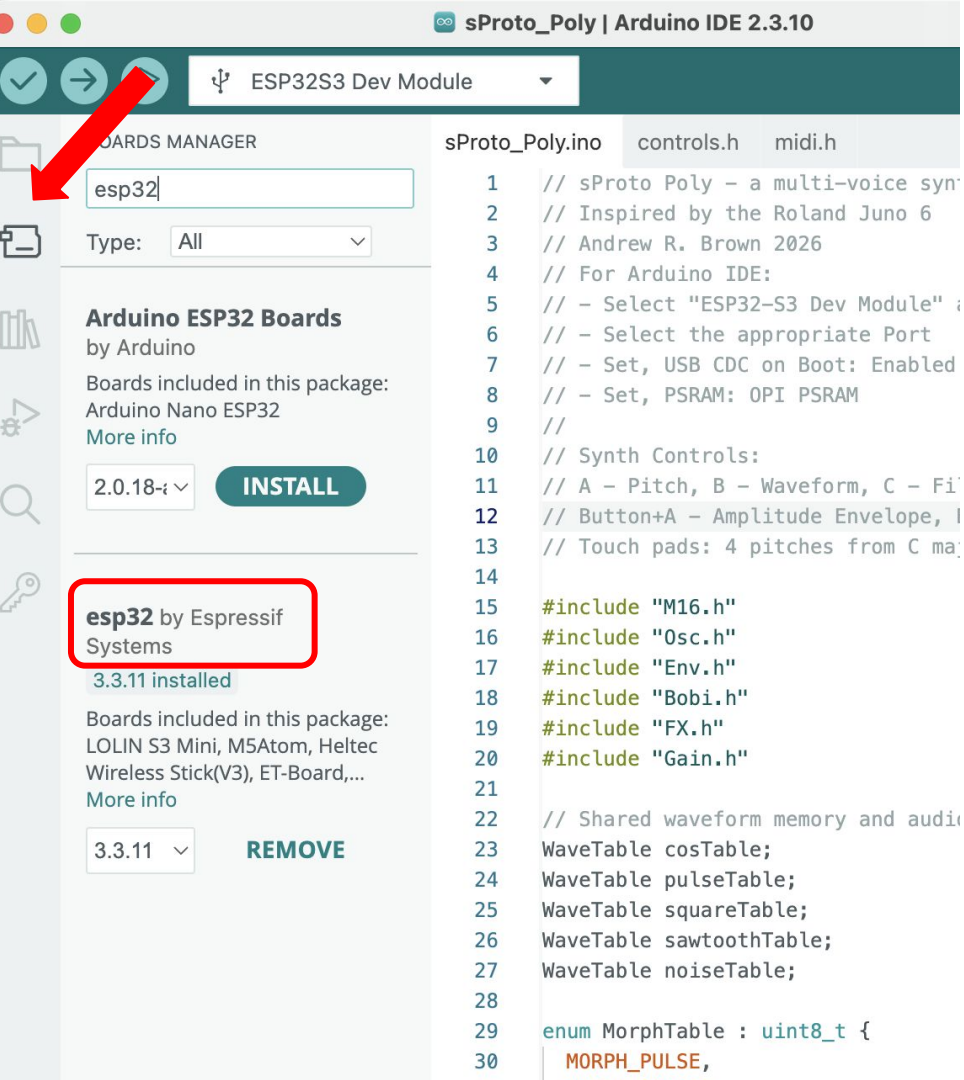


Add the ESP32 boards manager URL

Open Arduino IDE Settings

Google: installing esp32

Paste json link into
“Additional Boards Manager URLs”

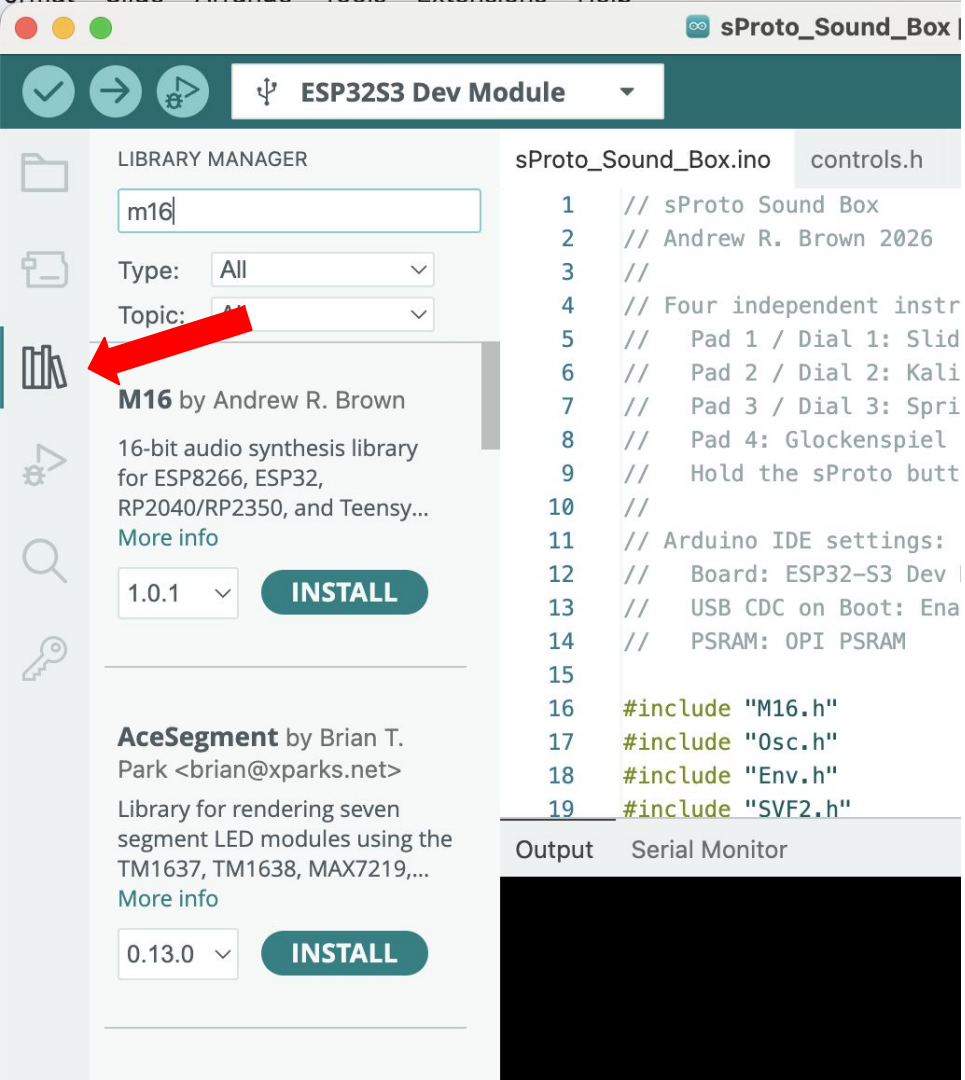


Add the ESP32 boards manager

Open Boards Manager (red arrow)

Locate ESP32 by Espressif

Click INTSALL



Install the libraries

Open the Library Manager (arrow)

M16

- https://github.com/algomusic/M16

MultiControl

- https://github.com/algomusic/MultiControl

Home

Edit New page

Andrew Brown edited this page on Aug 31 · 8 revisions

M16 Library API Reference

A 16-bit audio synthesis library for ESP8266, ESP32, RP2040/RP2350 (Pico), and Teensy 4.x using I2S DACs and ADCs.

Quick Start

```
#include "M16.h"
#include "Osc.h"

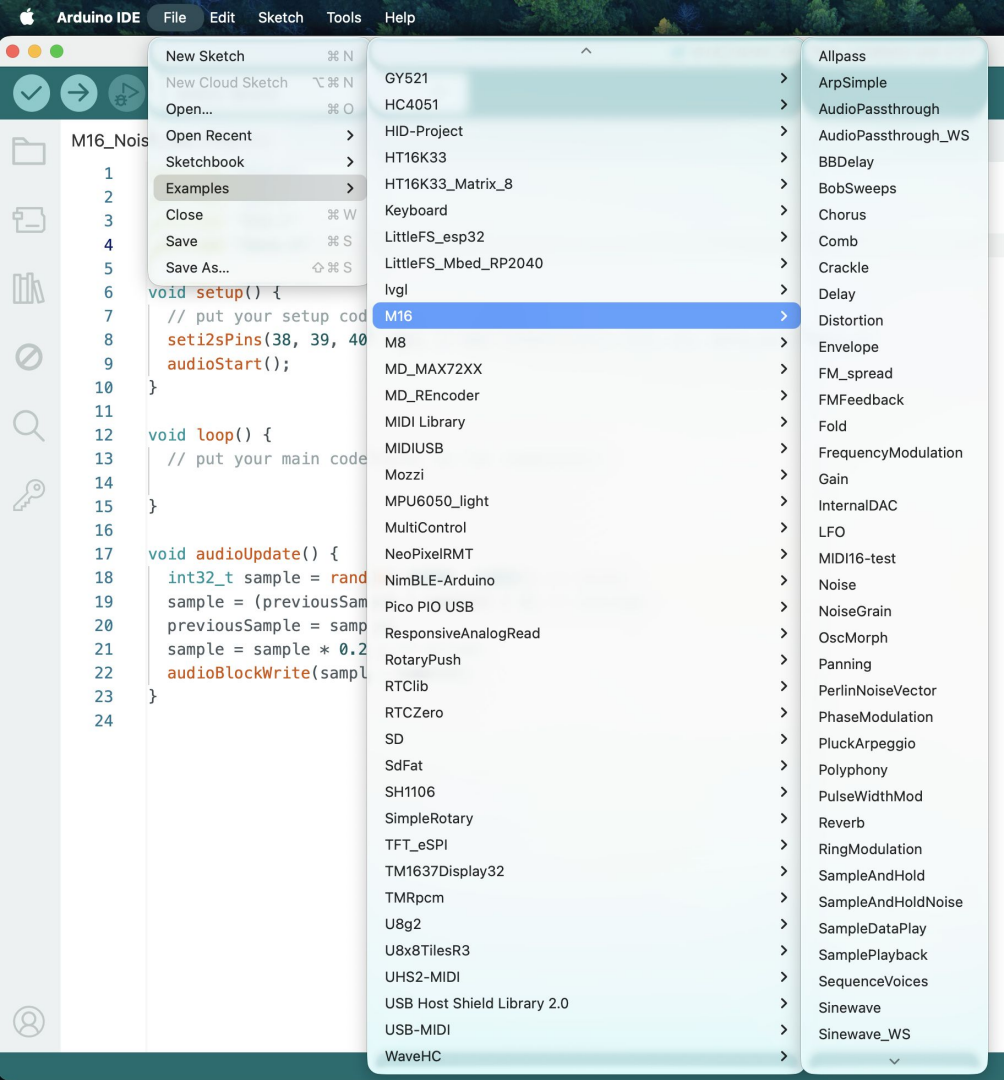
Osc osc;

void setup() {
  Serial.begin(115200);
  setIsDualCore(false);
  osc.sinGen();
  osc.setPitch(69); // A4
  audioStart();
}
```

Pages 1

Find a page or section...

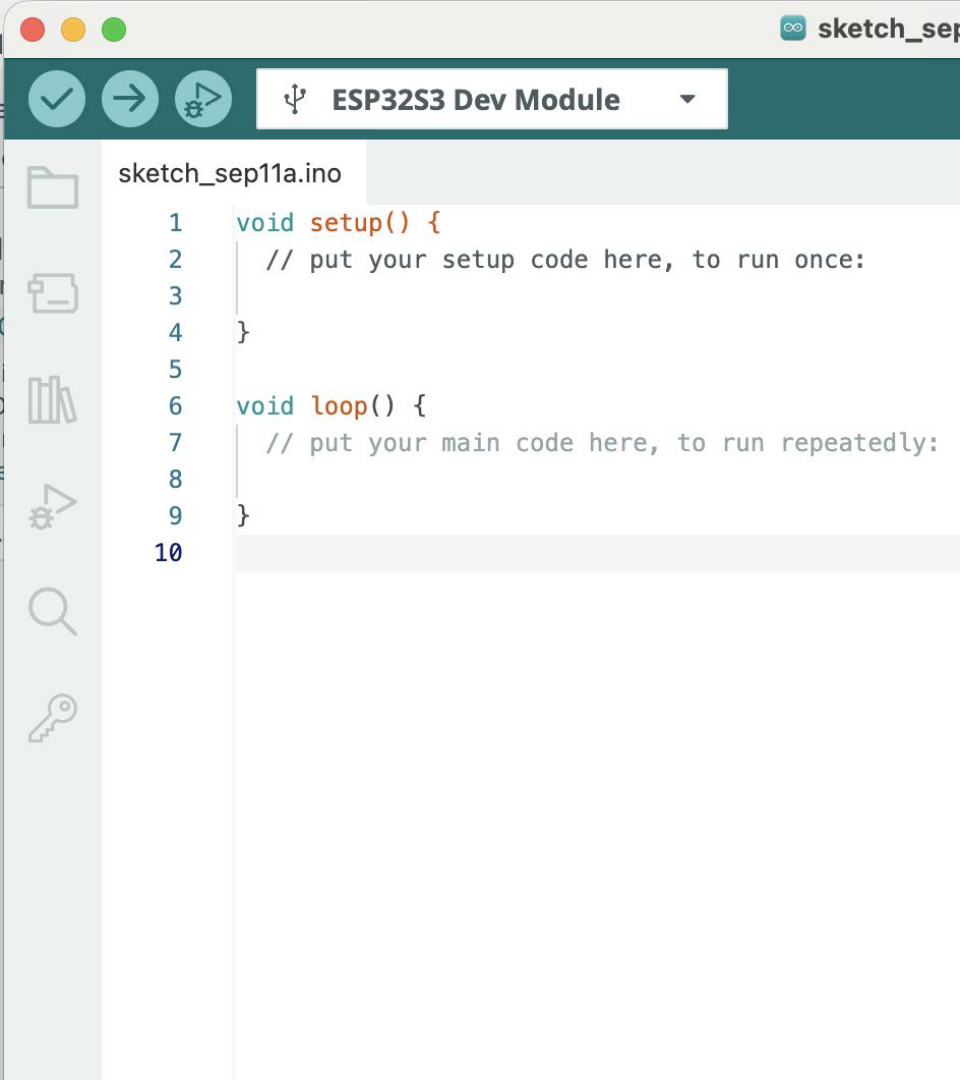
Home
- M16 Library API Reference
  - Quick Start
  - Core (M16.h)
    - Audio System
    - Platform Behaviour
    - Dual-Core Partition API
    - Pico Family
    - Utilities
  - Osc.h — Wavetable Oscillator
    - Generating a Table
    - Sharing Tables
    - Key Methods
    - Modulation



M16 Examples

There are many examples provided with the M16 library. Explore them to see what's possible and how to use various features.

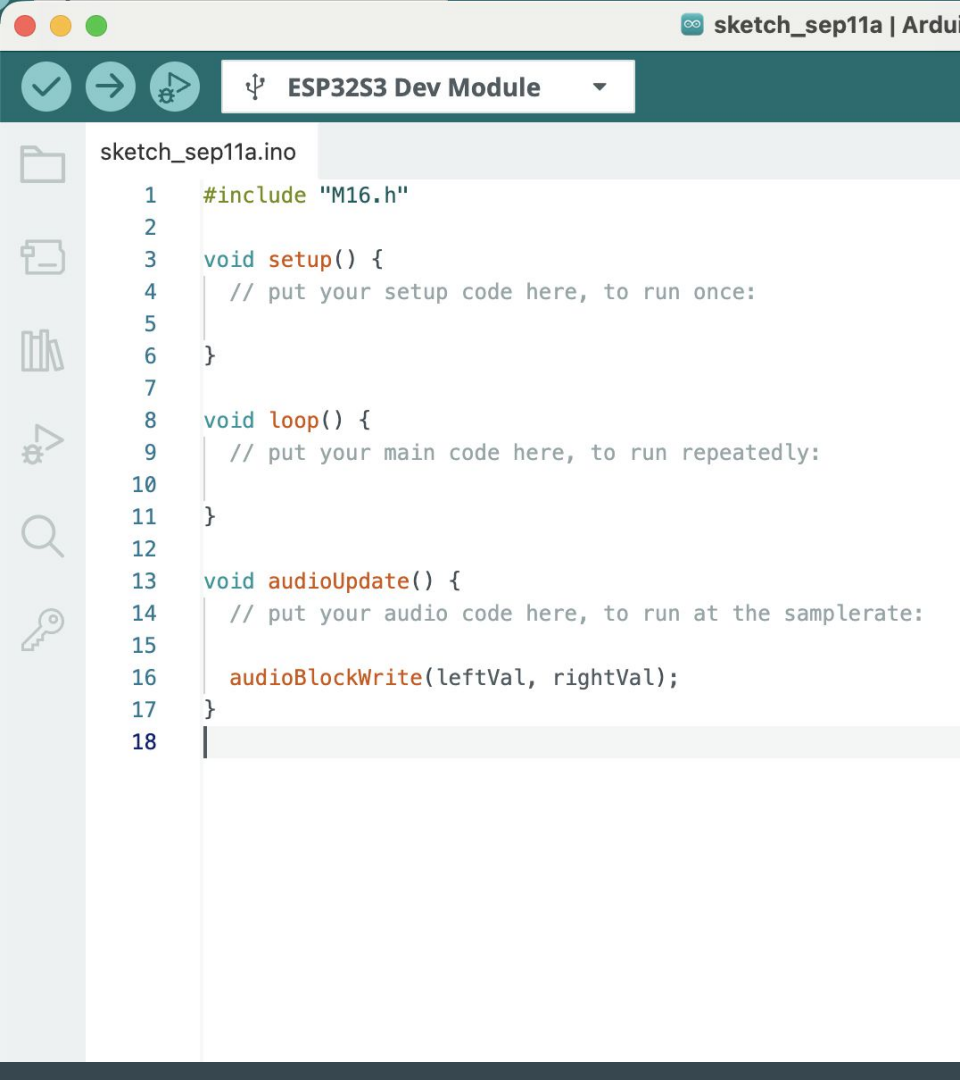
File -> Examples -> M16



New Sketch

```
void setup() {
  // put your setup code here, to run once:
}

void loop() {
  // put your main code here, to run repeatedly:
}
```



M16 Additions

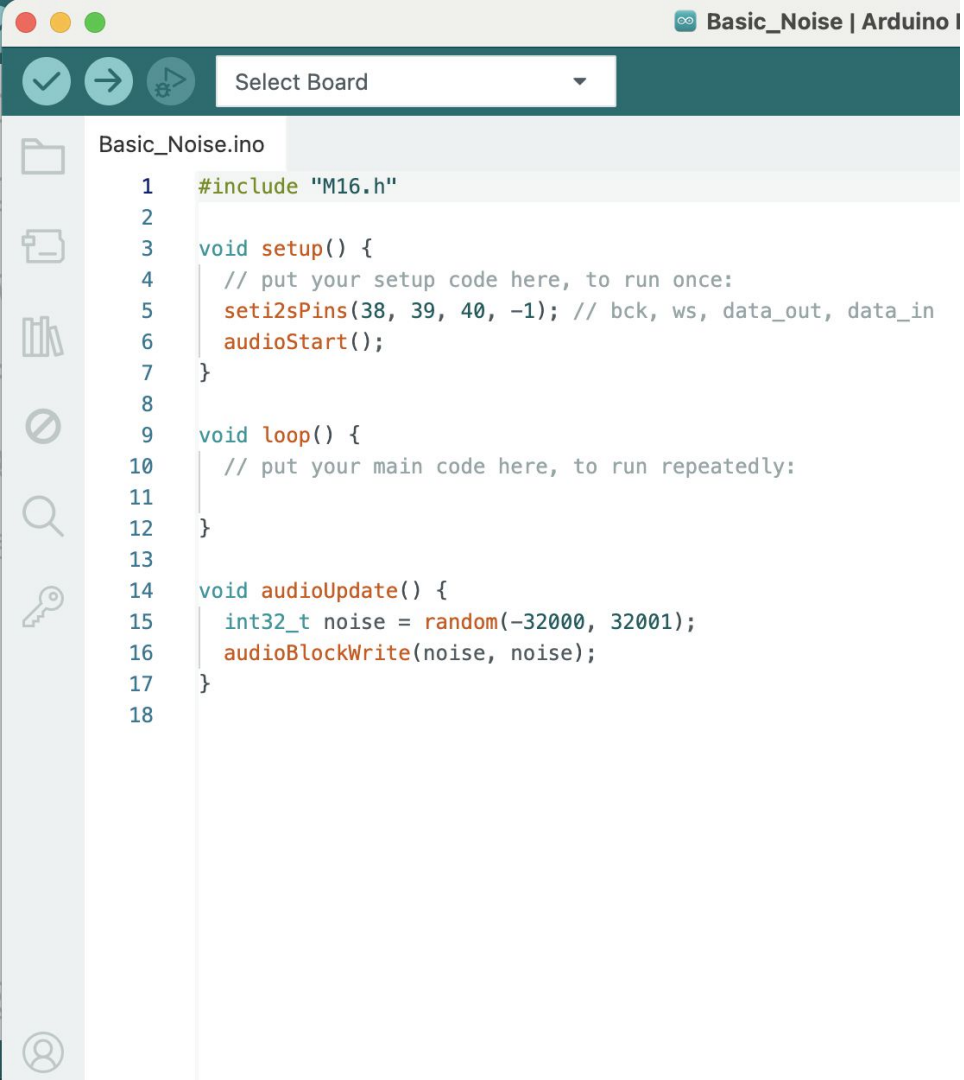
```
#include "M16.h"

void audioUpdate() {

  // put your audio code here, to run at the sample rate:

  audioBlockWrite(leftVal, rightVal);

}
```



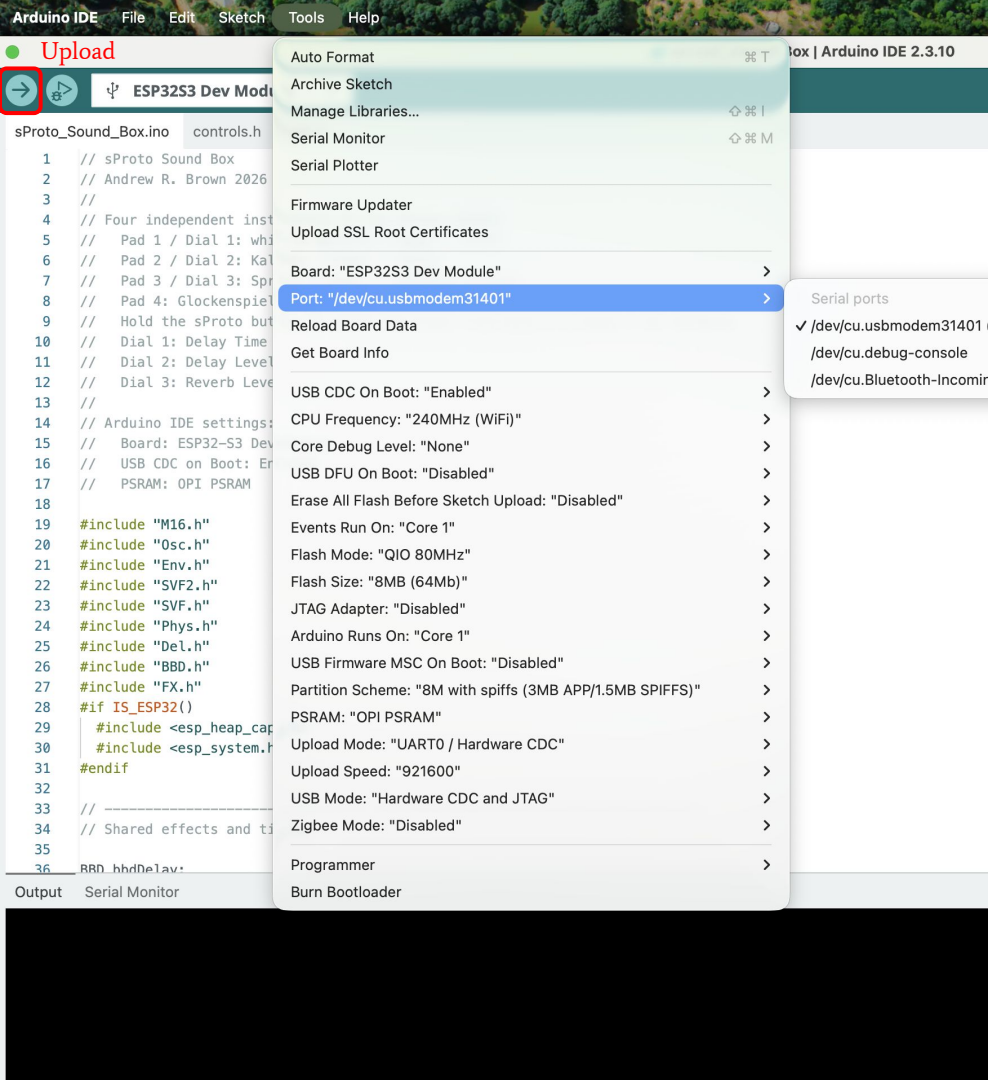
Basic Noise

Random values create noise

```
seti2sPins(bck, ws, d_out, d_in);

audioStart();

int32_t noise = random(-32000, 32001);
```



Uploading

Select Board & Port from Tools menu

Board:

“ESP32S3 Dev Module”

Port:

usbmodem (MacOS) or COM# (PC)

Click Upload (->)

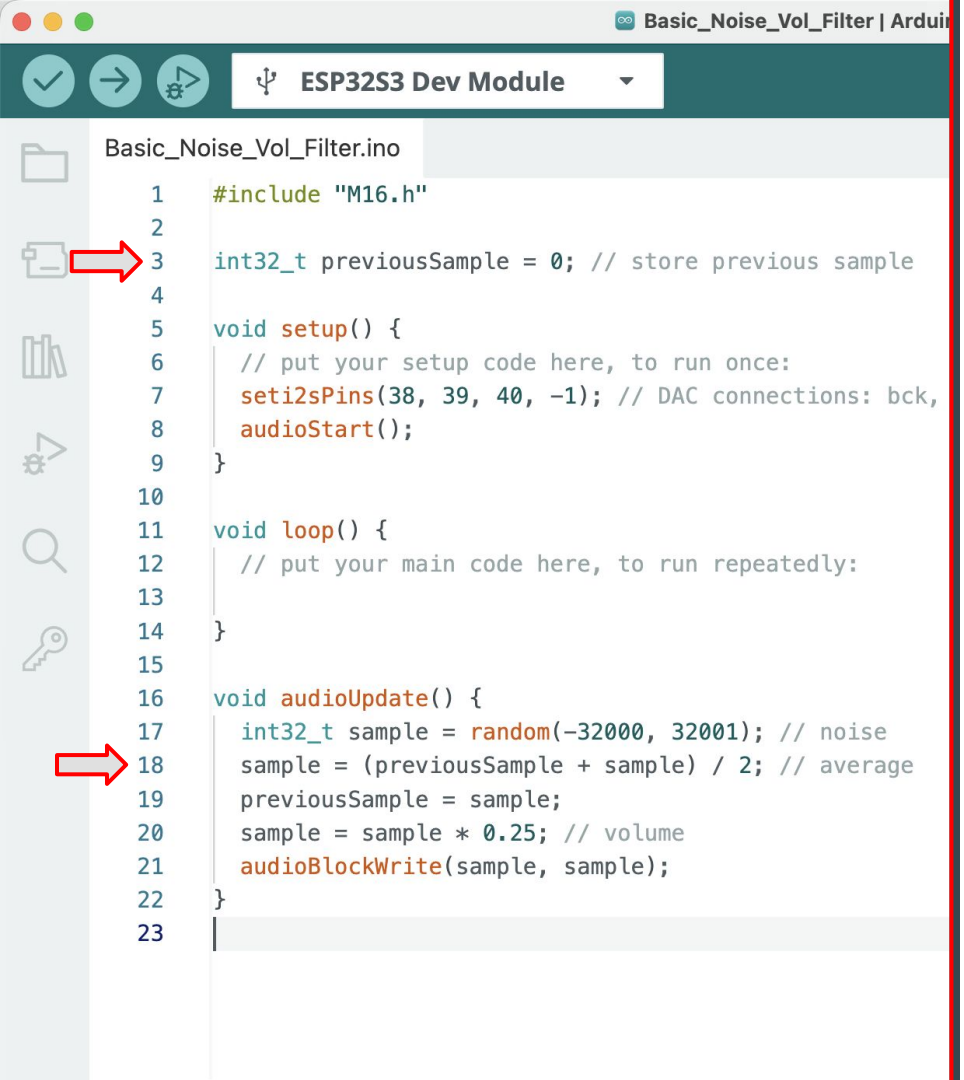


Basic Noise & Vol

Multiplication changes gain

```
int32_t noise = random(-32000, 32001);

sample = sample * 0.25;
```

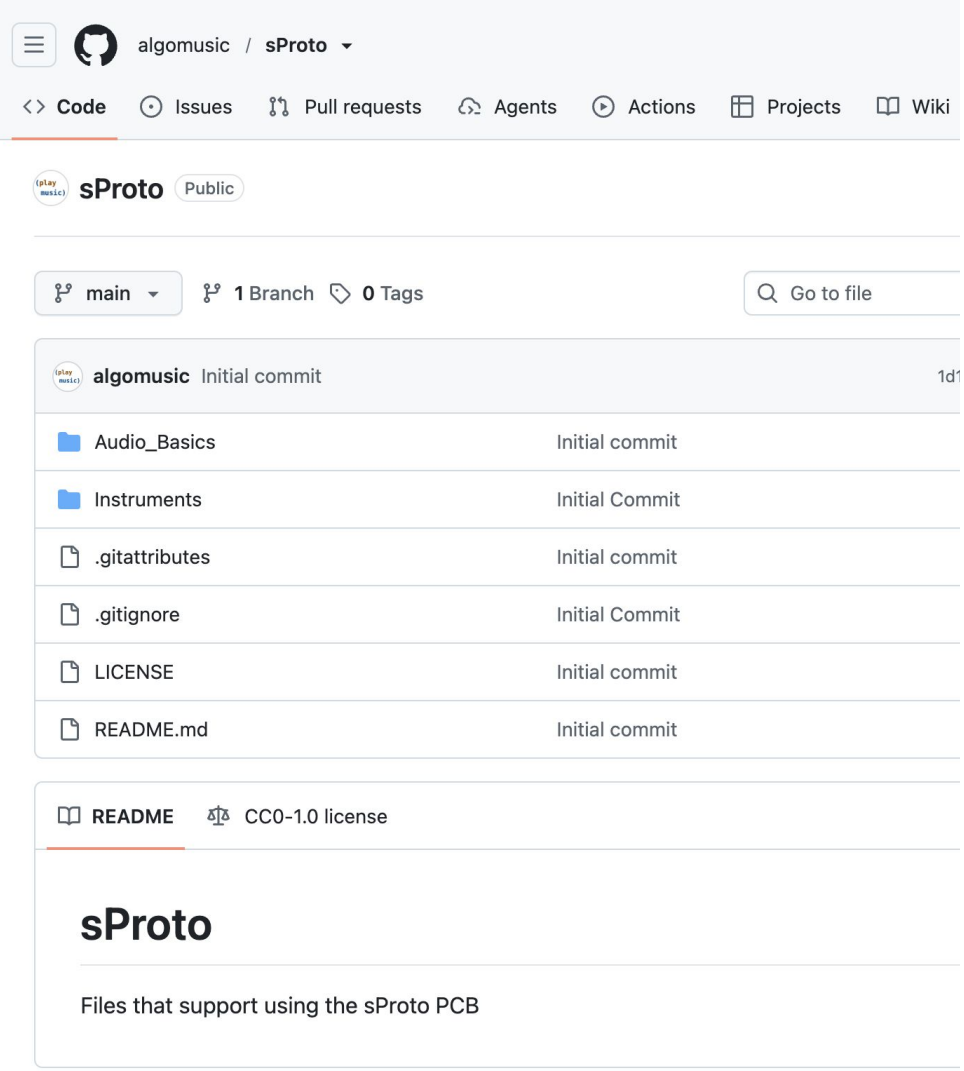


Noise & Filter & Vol

Averaging samples filters (LPF)

```
int32_t previousSample = 0;


int32_t noise = random(-32000, 32001);
sample = (previousSample + sample) / 2;
previousSample = sample;
sample = sample * 0.25;
```

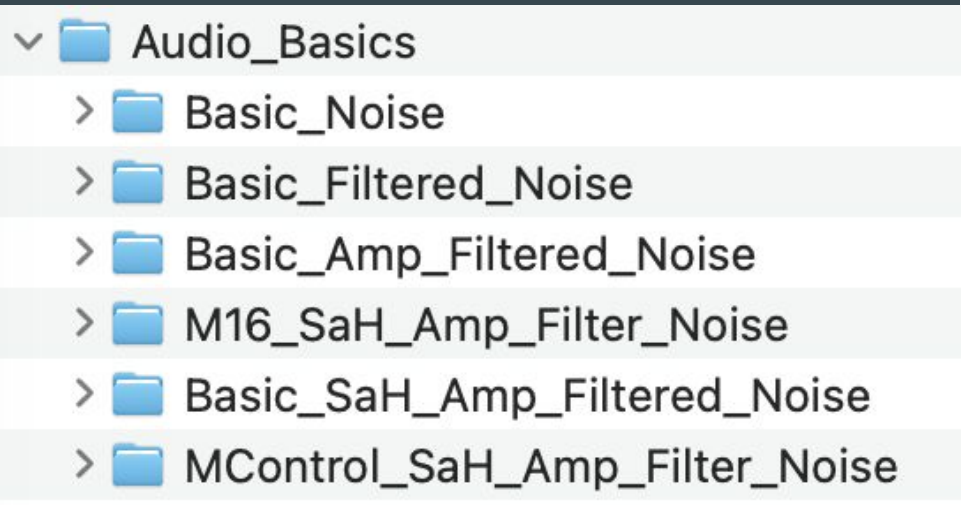


Files from GitHub

Access sProto files online:

https://github.com/algomusic/sproto

sProto -> Code -> Download Zip



Extending Noise

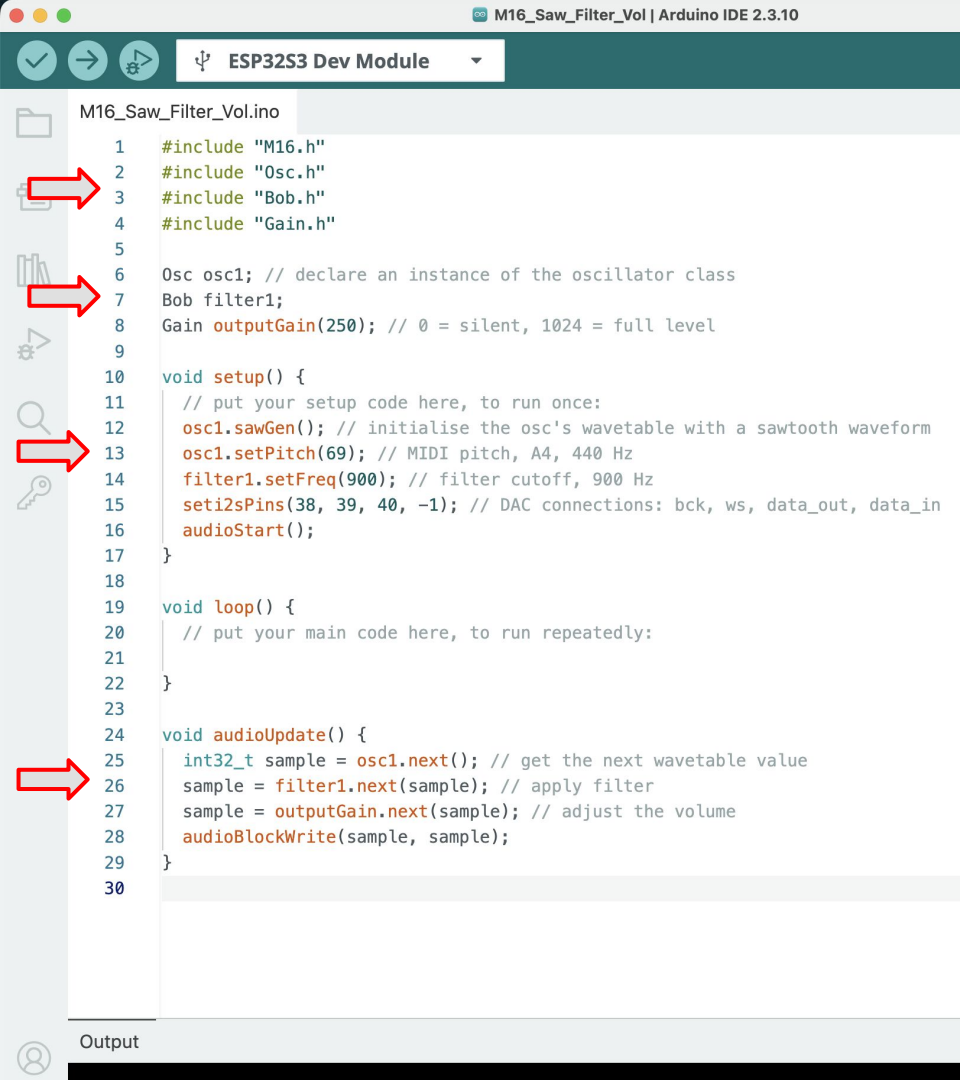
Filtering

Amplitude

S&H Noise

M16 classes

MultiControl



Saw & Filter & Vol

Using M16 for basic sound processing

- Import required M16 classes
- Create instances of them
- Apply settings
- Use them in audioUpdate()

MultiControl_Basics.ino

```
1   #include "MultiControl.h"
2
3   // sProto ESP32 Dev Module control pins
4   MultiControl dial1(5, 1); // GPIO, Potentiometer
5   MultiControl dial2(6, 1);
6   MultiControl dial3(7, 1);
7   MultiControl button(8, 2); // GPIO, Button
8
9   MultiControl pad1(1, 0); // GPIO, Touch
10  MultiControl pad2(2, 0);
11  MultiControl pad3(3, 0);
12  MultiControl pad4(4, 0);
13
14  void setup() {
15    Serial.begin(115200);
16    delay(500);
17    Serial.println("sProto MultiControl basics");
18  }
19
20  void loop() {
21    const int dial1Value = dial1.readPot();
22    const int dial2Value = dial2.readPot();
23    const int dial3Value = dial3.readPot();
24
```

MultiControl Print Values

Initialise with GPIO pin & type

MC Types:

0 = Touch

1 = Potentiometer

2 = Button

Access values:

readPot() - int

isPressed() - boolean

isTouched() - boolean

MC_M16_Saw_Filter_Vol.ino

```
#include "M16.h"
#include "Osc.h"
#include "Bob.h"
#include "Gain.h"

Osc osc1; // declare an instance of the oscillator class
Bob filter1;
Gain outputGain(250); // 0 = silent, 1024 = full level

#include "MultiControl.h"

MultiControl dial1(5, 1); MultiControl dial2(6, 1);
MultiControl dial3(7, 1); MultiControl button(8, 2);
MultiControl pad1(1, 0); MultiControl pad2(2, 0);
MultiControl pad3(3, 0); MultiControl pad4(4, 0);

void setup() {
  osc1.sawGen(); // initialise the osc's wavetable with a sawtooth waveform
  osc1.setPitch(69); // MIDI pitch, A4, 440 Hz
  filter1.setFreq(900); // filter cutoff, 900 Hz
  seti2sPins(38, 39, 40, -1); // DAC connections: bck, ws, data_out, data_in
  audioStart();
}

void loop() {
  const int dial1Value = dial1.readPotChanged(); // oscillator pitch
  if (dial1Value >= 0) osc1.setPitch(map(dial1Value, 0, 1024, 24, 96));
  const int dial2Value = dial2.readPotChanged(); // filter cutoff
  if (dial2Value >= 0) filter1.setFreq(map(dial2Value, 0, 1024, 200, 5000));
  const int dial3Value = dial3.readPotChanged(); // volume
  if (dial3Value >= 0) outputGain.setLevel(dial3Value);

  if (pad1.isTouched()) osc1.setPitch(60);
  if (pad2.isTouched()) osc1.setPitch(62);
  if (pad3.isTouched()) osc1.setPitch(64);
  if (pad4.isTouched()) osc1.setPitch(65);
}

void audioUpdate() {
  int32_t sample = osc1.next(); // get the next wavetable value
  sample = filter1.next(sample); // apply filter
  sample = outputGain.next(sample); // adjust the volume
  audioBlockWrite(sample, sample);
}
```

MC Saw & Filter & Vol

Add MultiControl and Synth together

- Initialise M16 synth elements
- Initialise MultiControl elements
- Assign interface changes to vals
- Assign pad triggers to pitches

MC_M16_Saw_Filter_Env.ino

```
#include "M16.h"
#include "Osc.h"
#include "Bob.h"
#include "Env.h"
#include "Gain.h"

Osc osc1; // declare an instance of the oscillator class
Bob filter1;
Env ampEnv1;
Gain outputGain(250); // 0 = silent, 1024 = full level

#include "MultiControl.h"

MultiControl dial1(5, 1); MultiControl dial2(6, 1);
MultiControl dial3(7, 1); MultiControl button(8, 2);
MultiControl pad1(1, 0); MultiControl pad2(2, 0);
MultiControl pad3(3, 0); MultiControl pad4(4, 0);

void setup() {
  osc1.sawGen(); // initialise the osc's wavetable with a sawtooth waveform
  osc1.setPitch(69); // MIDI pitch, A4, 440 Hz
  filter1.setFreq(900); // filter cutoff, 900 Hz
  seti2sPins(38, 39, 40, -1); // DAC connections: bck, ws, data_out, data_in
  audioStart();
}

void loop() {
  const int dial1Value = dial1.readPotChanged(); // oscillator pitch
  if (dial1Value >= 0) osc1.setPitch(map(dial1Value, 0, 1024, 24, 96));
  const int dial2Value = dial2.readPotChanged(); // filter cutoff
  if (dial2Value >= 0) filter1.setFreq(map(dial2Value, 0, 1024, 200, 5000));
  const int dial3Value = dial3.readPotChanged(); // volume
  if (dial3Value >= 0) outputGain.setLevel(dial3Value);

  if (pad1.isTouched()) {
    osc1.setPitch(60);
    ampEnv1.start();
  }
  if (pad2.isTouched()) {
    osc1.setPitch(62);
    ampEnv1.start();
  }
  if (pad3.isTouched()) {
    osc1.setPitch(64);
    ampEnv1.start();
  }
  if (pad4.isTouched()) {
    osc1.setPitch(65);
    ampEnv1.start();
  }
}

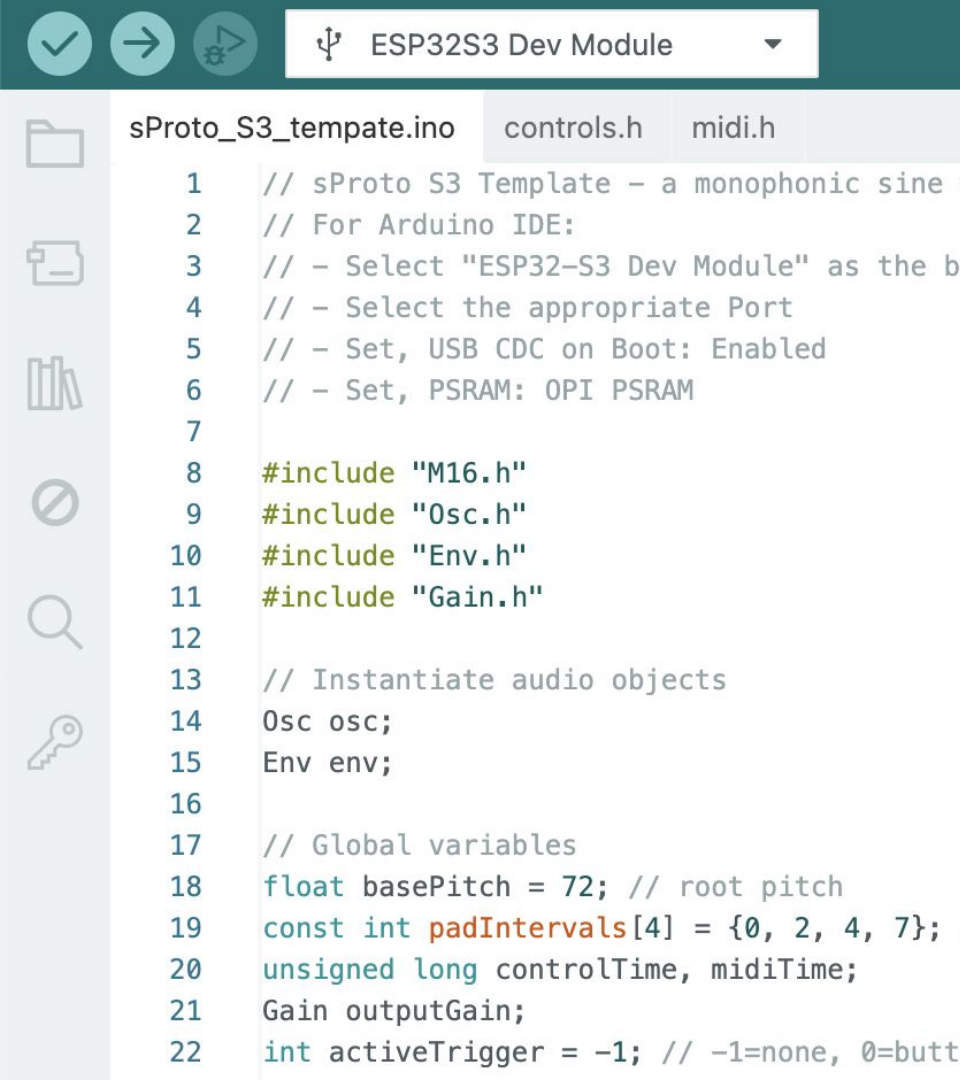
void audioUpdate() {
  int32_t sample = osc1.next(); // get the next wavetable value
  sample = filter1.next(sample); // apply filter
  sample = (ampEnv1.next() * sample) >> 16;
  sample = outputGain.next(sample); // adjust the volume
  audioBlockWrite(sample, sample);
}
```

Add an Envelope

Apply an amplitude envelope to notes

- Import the Env.h class
- Create an envelope object
- Add a .start() when triggered
- Process sound through the env

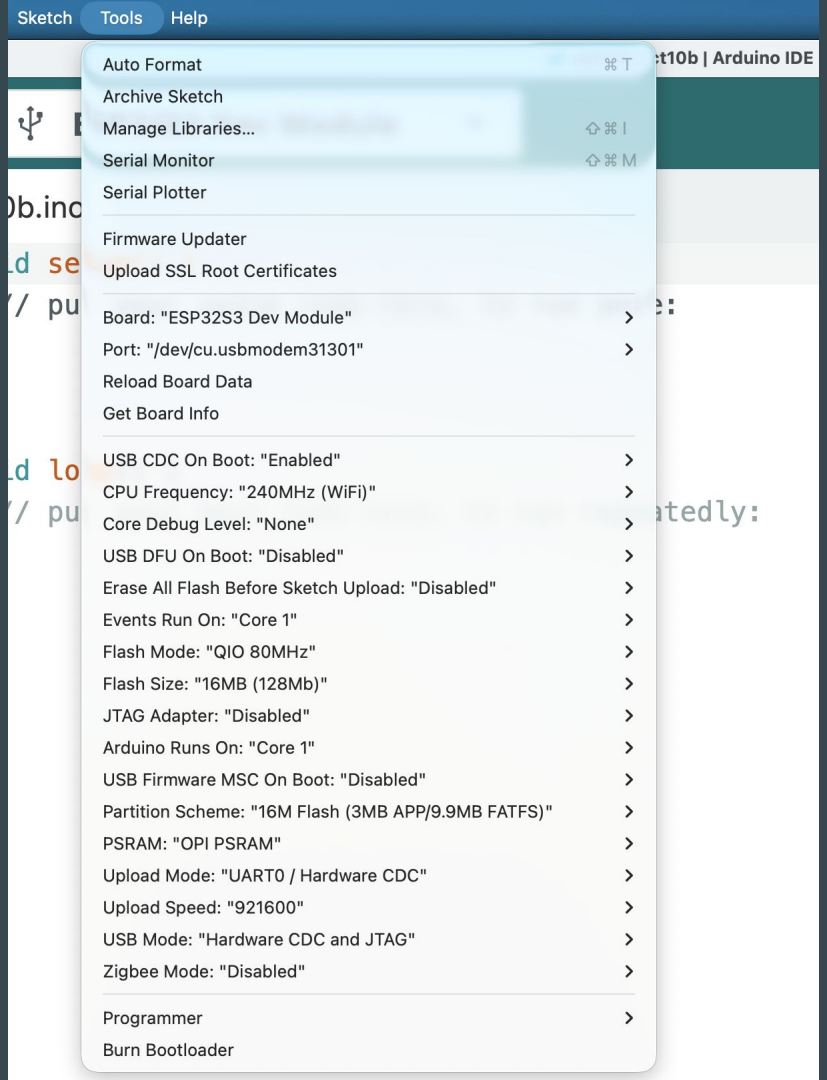
Env values are 16 bit integers and so sample multiplications need rescaling. “>> 16” is bit shift division to do this rescaling.



sProto S3 Template

3 Files:

- template.ino
- controls.h
- midi.h



Arduino Settings

Some sketches needs as much memory as it can get, so ensure these board settings are set in the Tools menu.

Flash Size: 16 MB

Partition Scheme: 16 M Flash

PSRAM: “OPI PSRAM”

Music Box Instruments

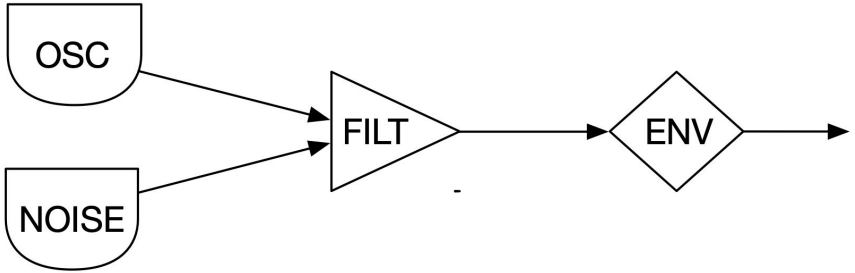
- Tin Whistle
- Glockenspiel
- Kalimba
- Spring



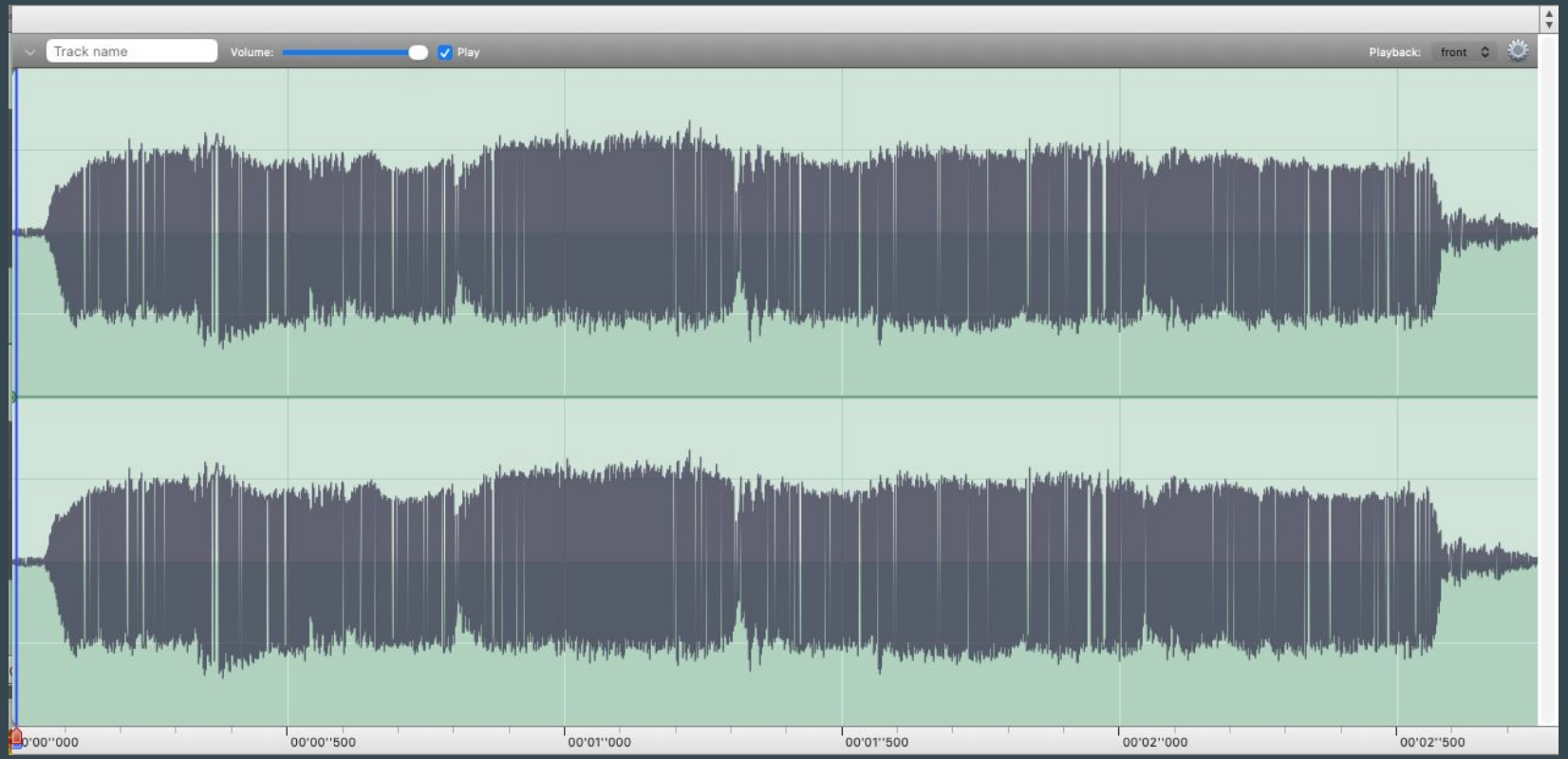


Tin Whistle

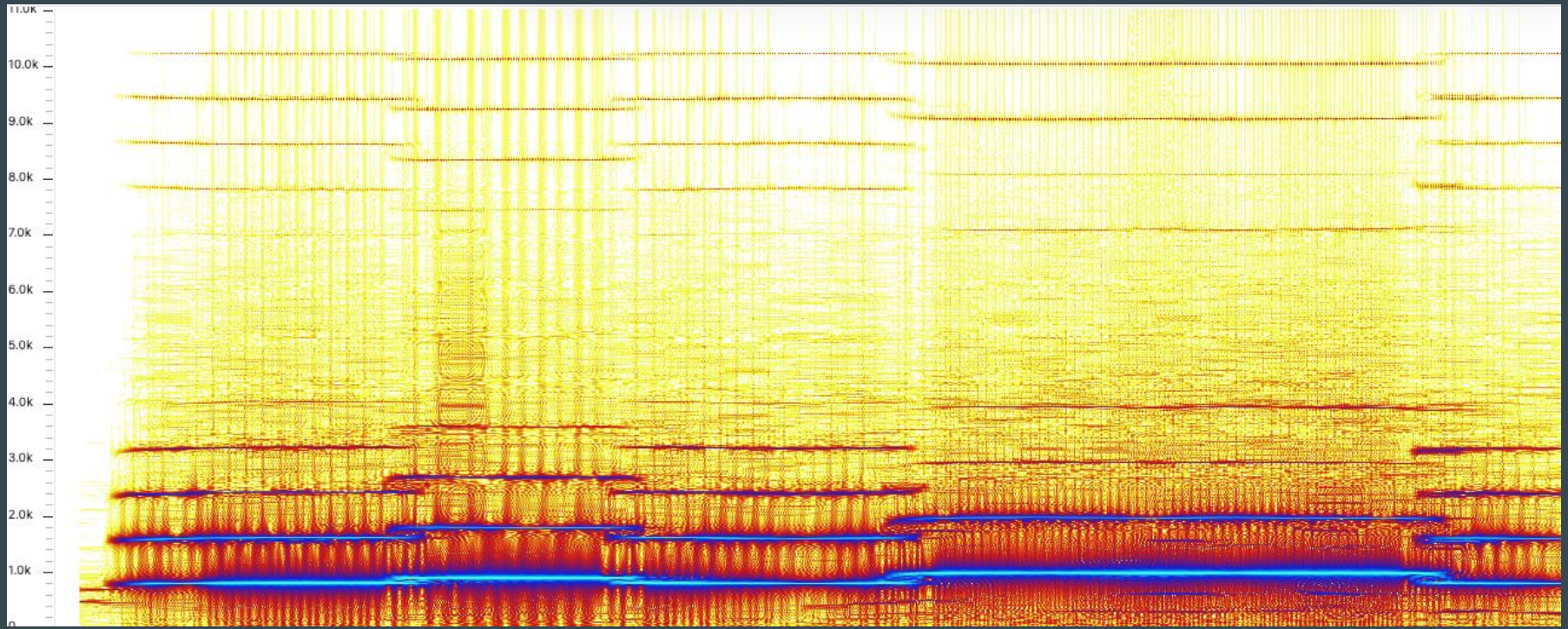
Subtractive Synthesis



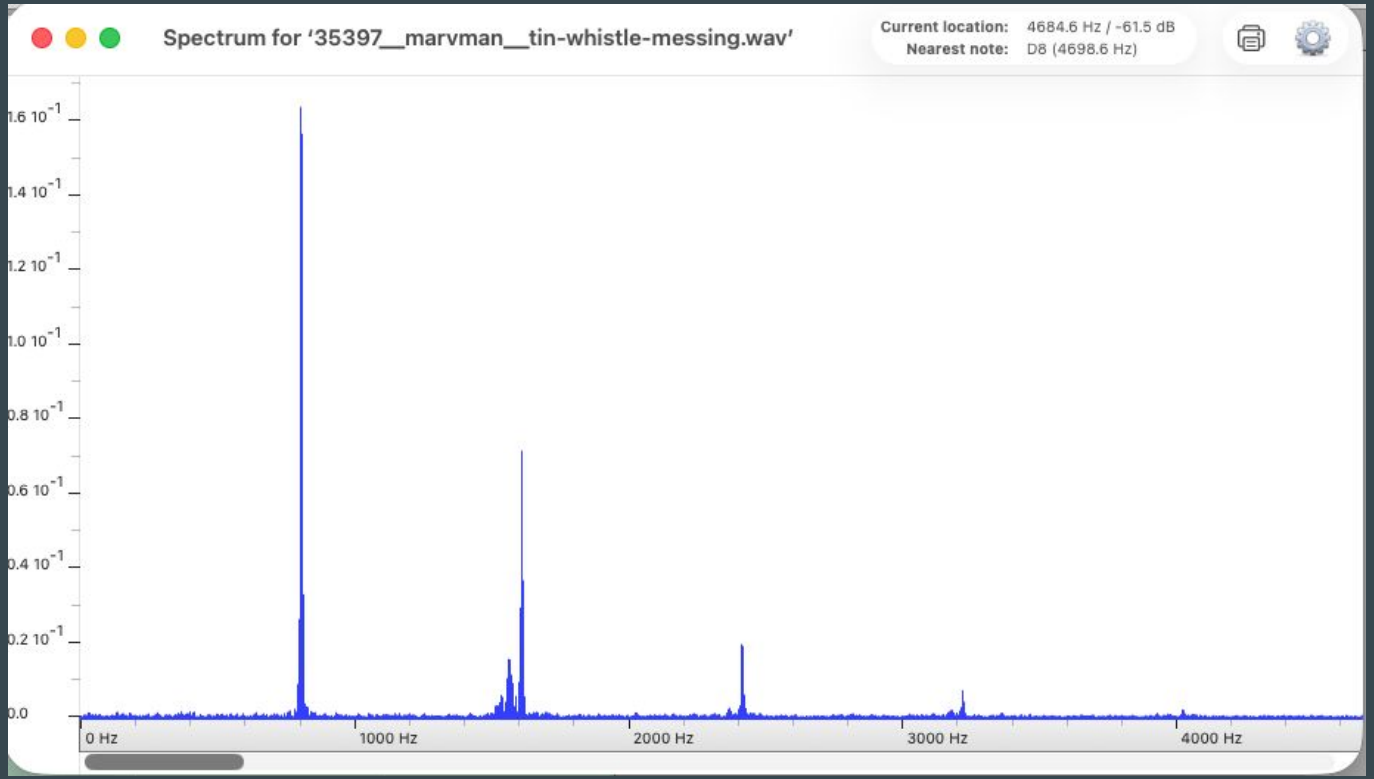
Tin Whistle Waveform



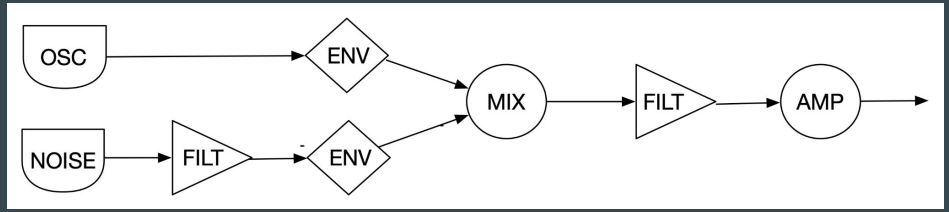
Tin Whistle Sonogram



Tin Whistle Spectrum

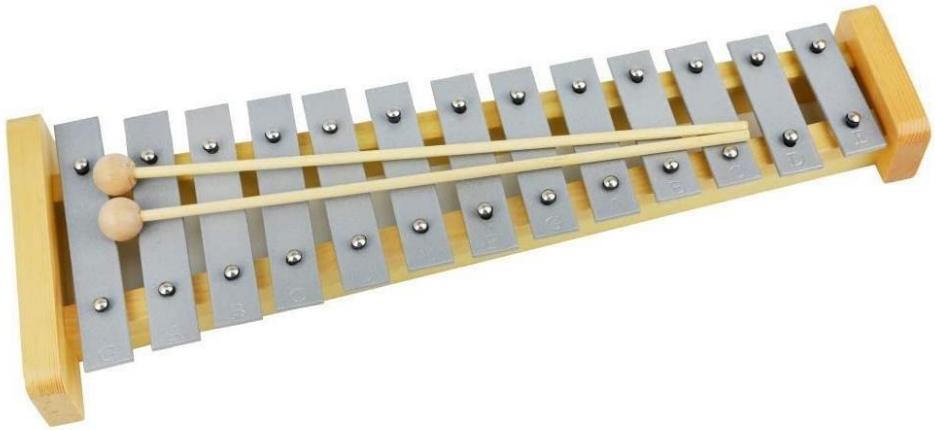


Tin Whistle synthesis function

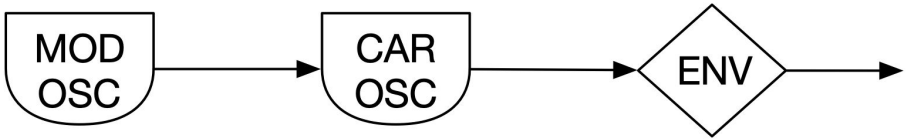



```
/* The audioUpdate function is required in all M16 programs. */
void audioUpdate() {
  int32_t tone = osc.phMod(slideVibratoLfo.next(), slideVibratoPhaseIndex); // tone with vibrato
  tone = (tone * toneAmpEnv.next()) >> 16; // apply amp env
  int32_t wind = noise.next(); // generate noise
  wind = noiseBPFilter.nextBPF(wind); // bandpass filter the noise
  wind = (wind * noiseAmpEnv.next()) >> 16; // amplitude envelope
  int32_t out = clip16(tone + wind); // mix tone and noise
  out = toneHPFilter.nextHPF(toneLPFilter.nextLPF(out)); // lowpass filter the mixed signal
  out = outputGain.next(out); // volume level
  audioBlockWrite(out, out); // send to DAC
}
```

Glockenspiel



Frequency Modulation Synthesis



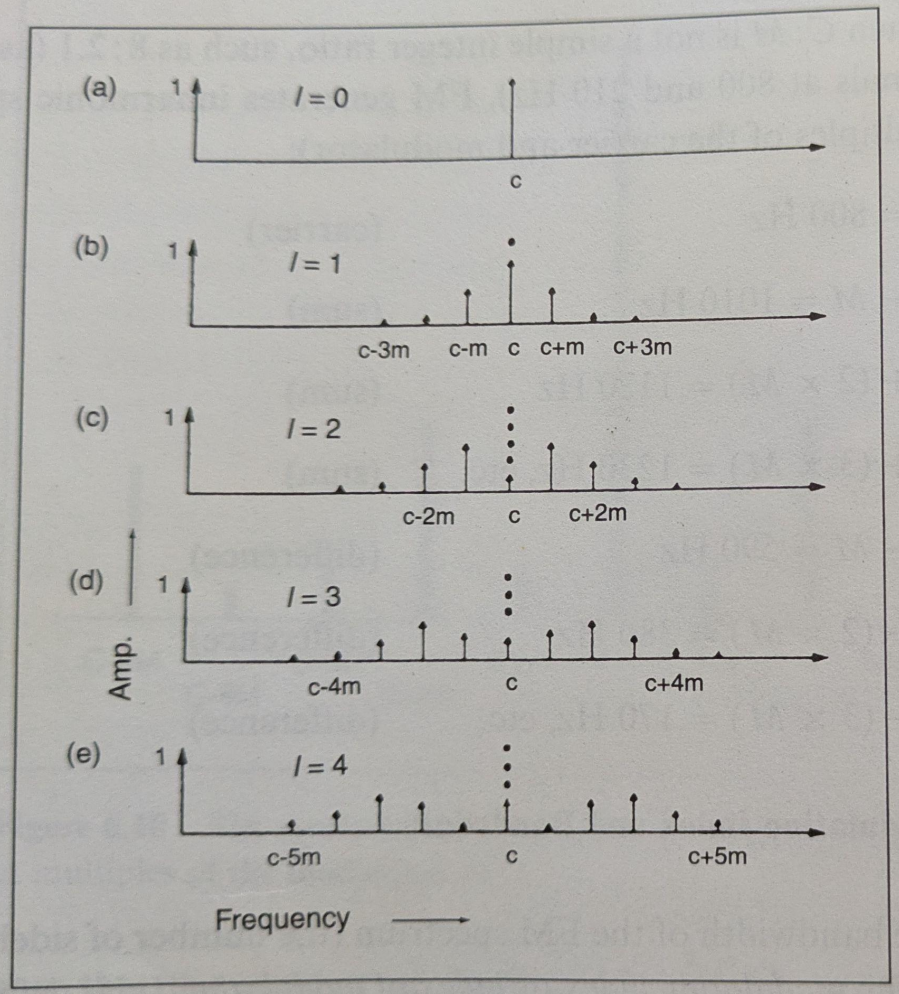


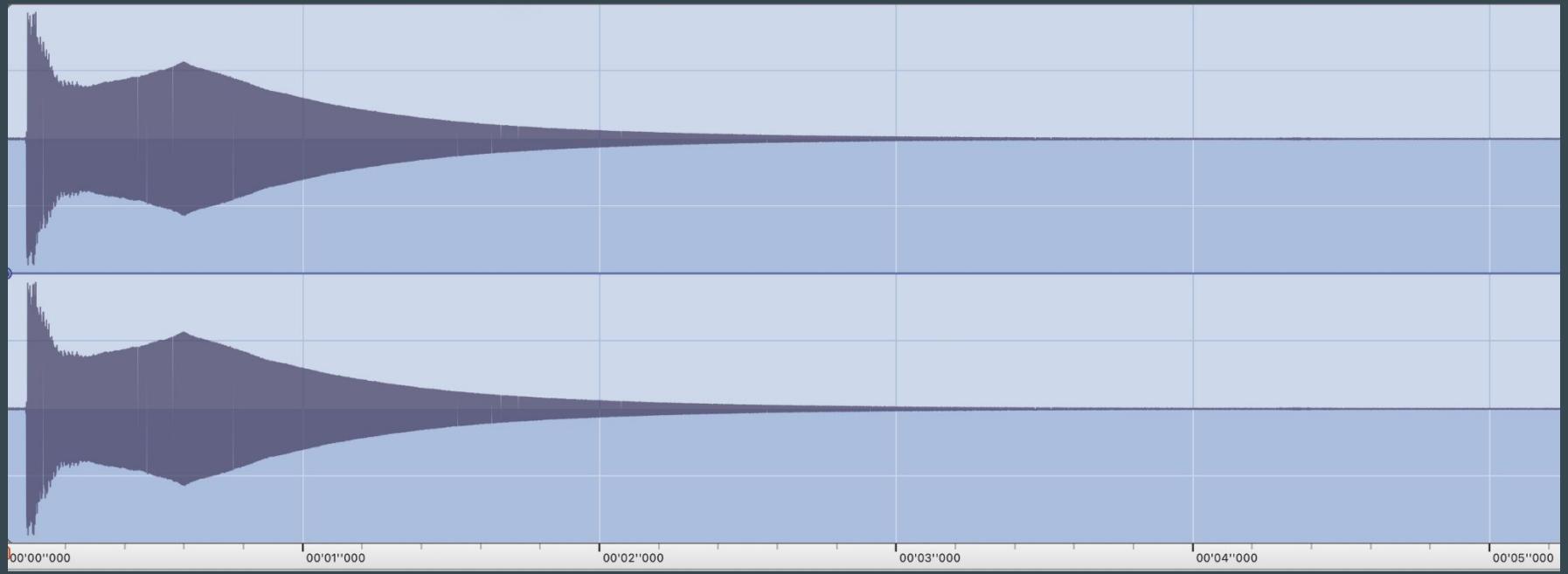
Figure 6.11 FM spectrum with increasing modulation index.

FM Spectrum

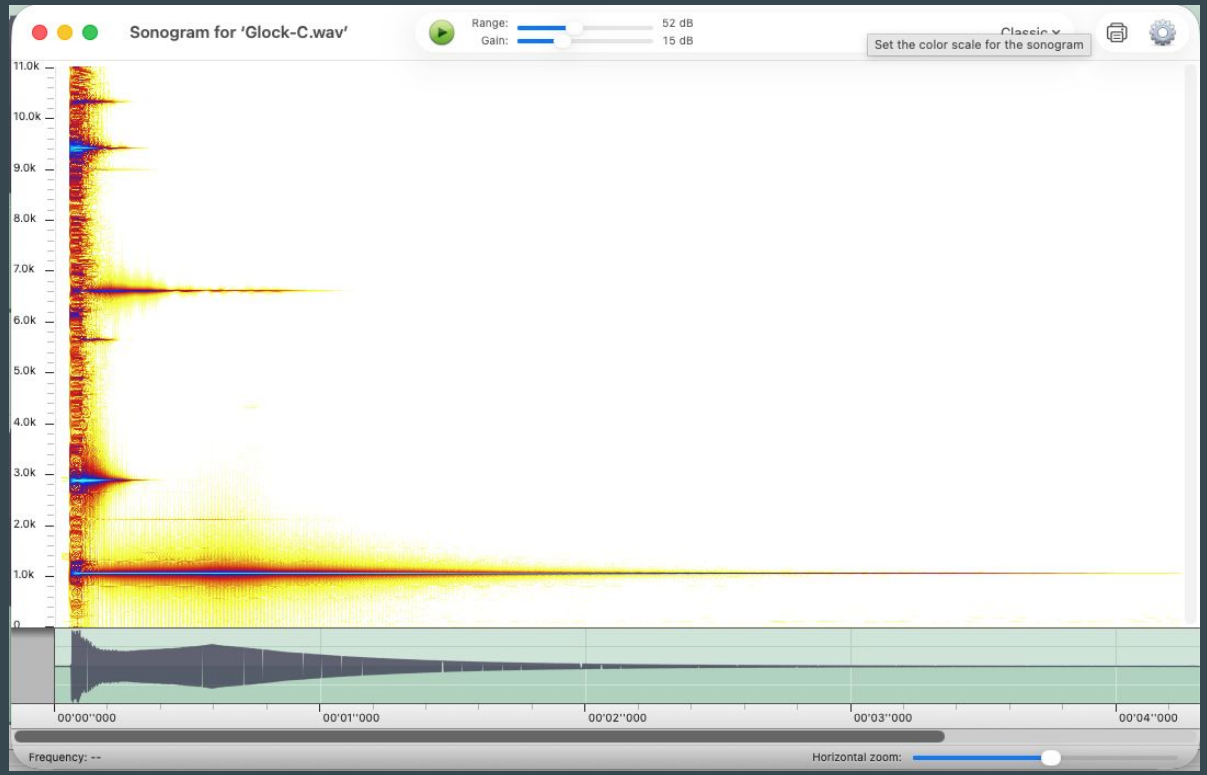
Sidebands at intervals of the modulating frequency symmetrical around the carrier frequency.

Roads, C. 1996. Computer Music Tutorial, The MIT Press.

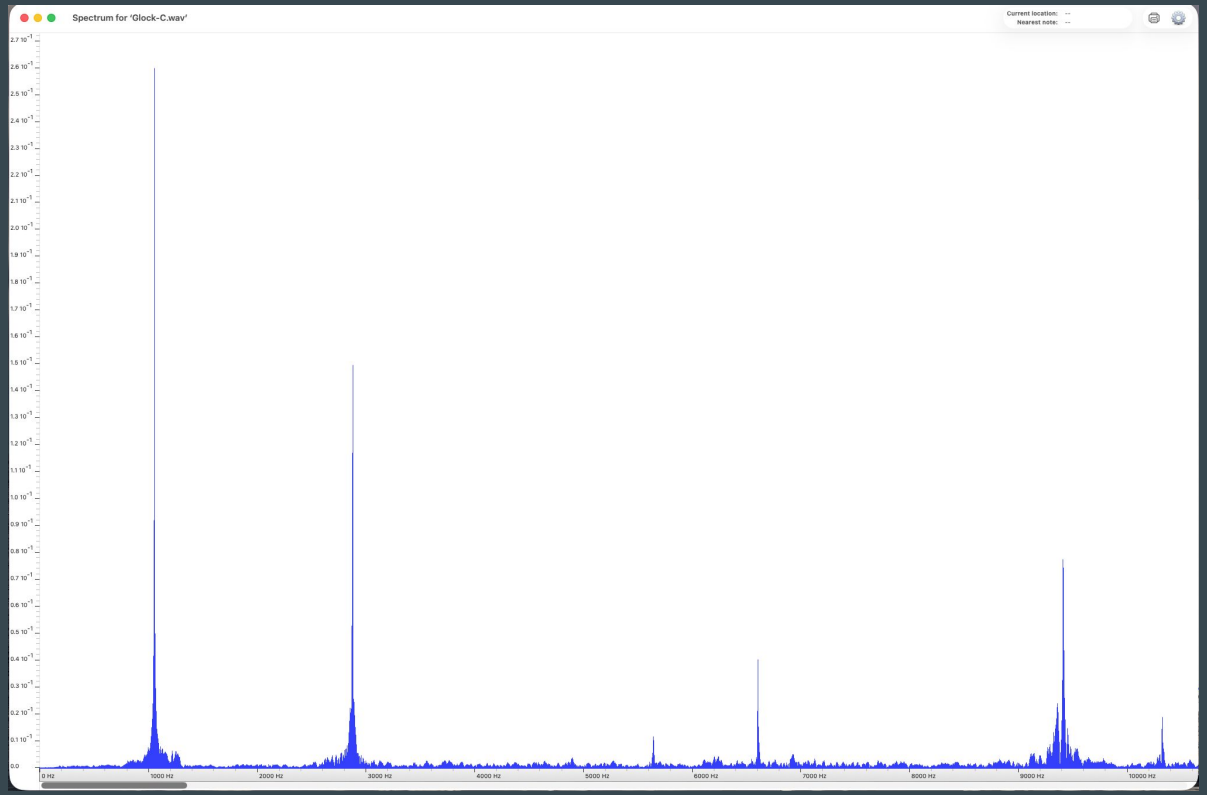
Glock Waveform



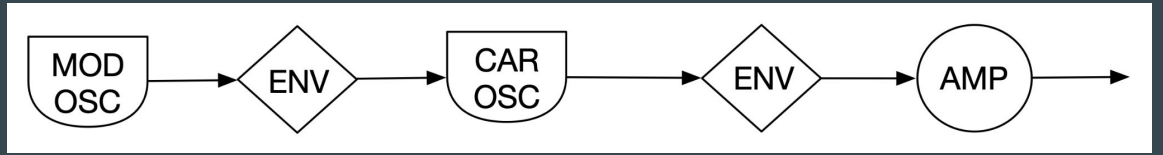
Glock Sonogram



Glock Spectrum



Glock synthesis function

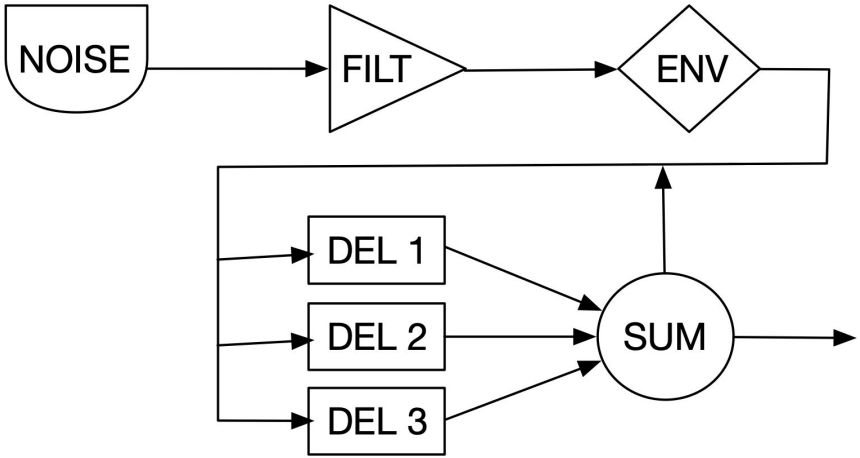



```
/* The audioUpdate function is required in all M16 programs. */
void audioUpdate() {
  int32_t modTone = modOsc.next(); // generate modulator
  int32_t modAmpEnv = (modTone * modEnv.next()) >> 16; // shape modulator
  int32_t tone = carrierOsc.phMod(modAmpEnv, modIndex); // generate modulated carrier
  int32_t toneAmpEnv = (tone * carrierEnv.next()) >> 16; // shape the carrier
  int32_t out = outputGain.next(toneAmpEnv); // volume
  audioBlockWrite(out, out); // send to DAC, same signal to L & R
}
```

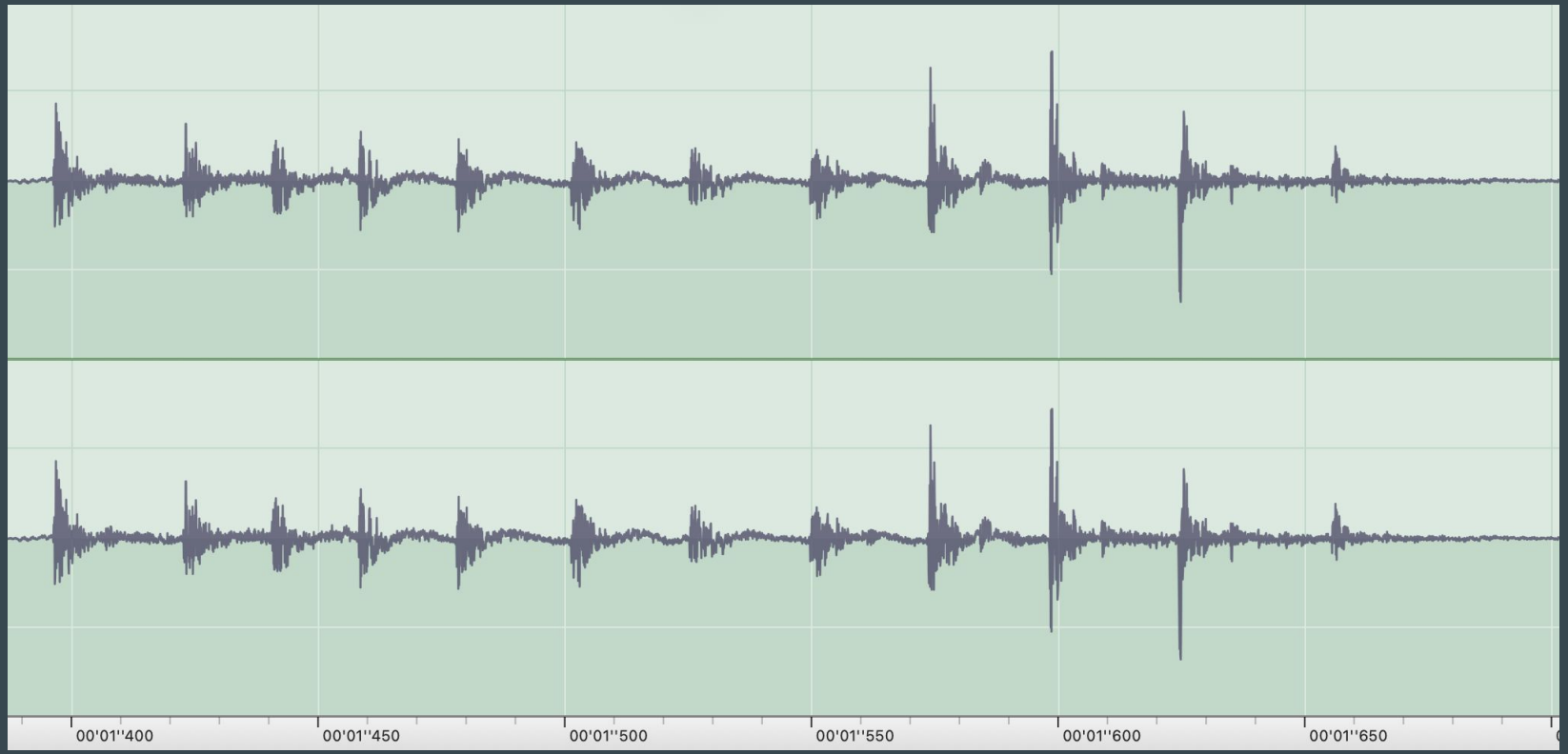


Spring

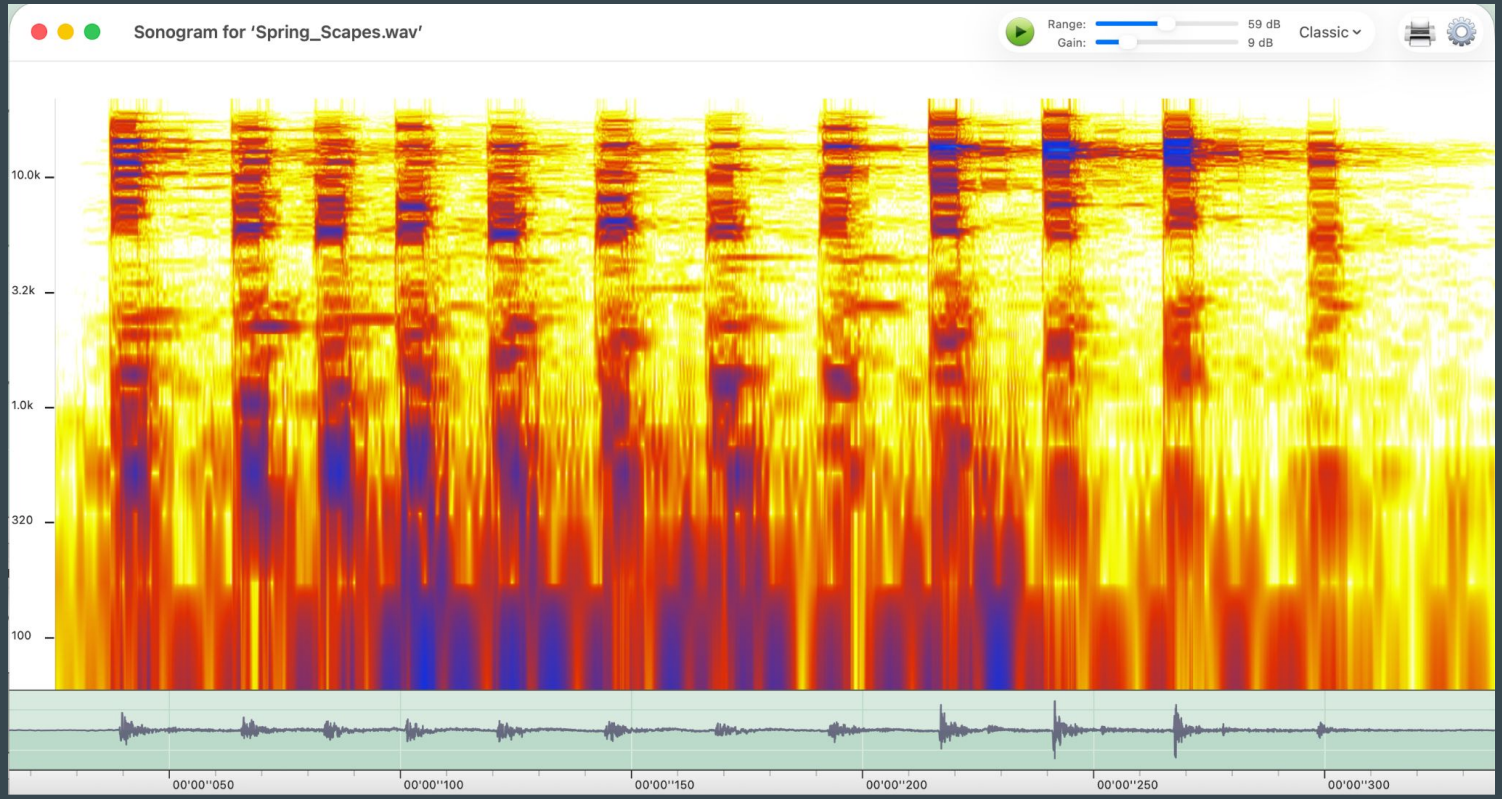
Excited Feedback Delays



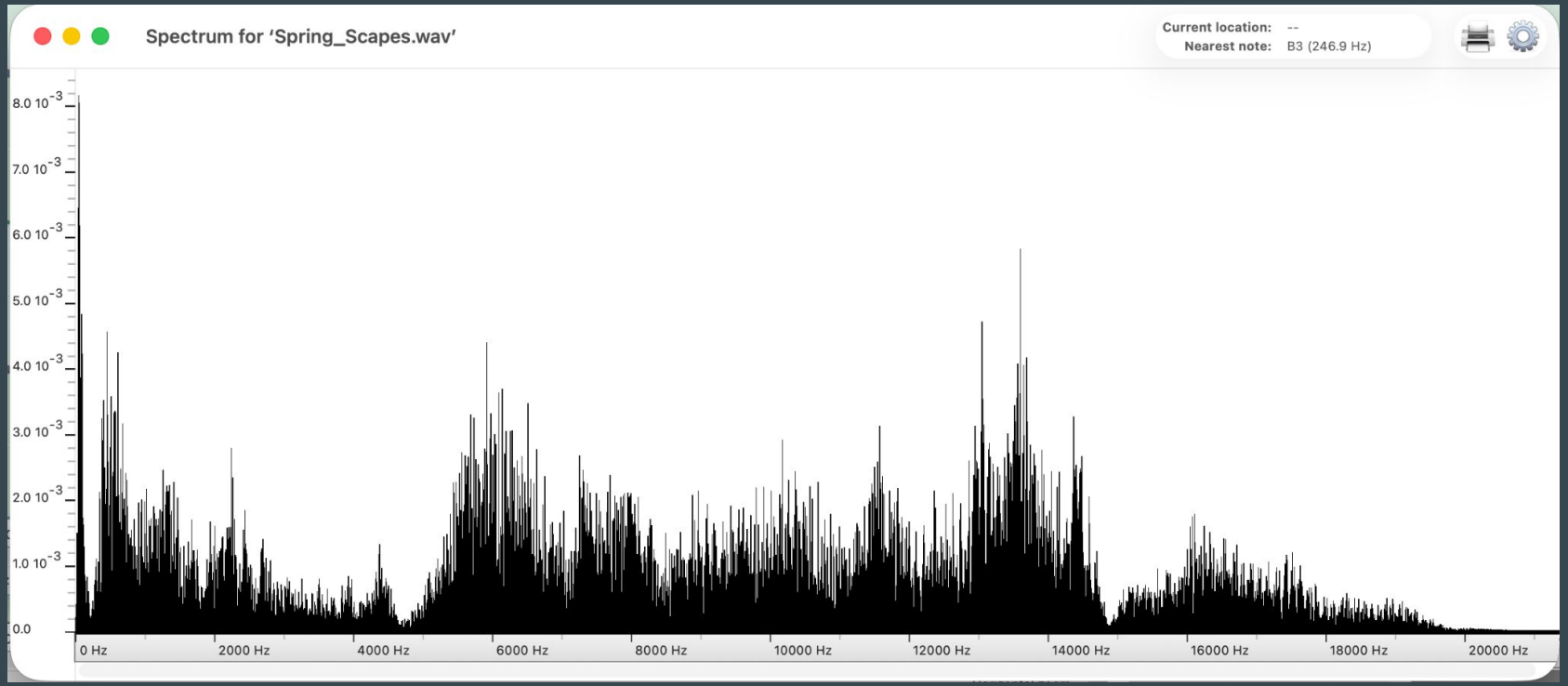
Spring Scrape Waveform



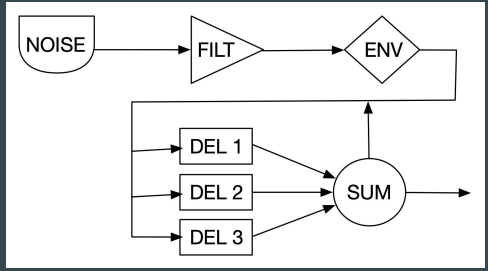
Spring Sonogram



Spring Spectrum



Spring synthesis function



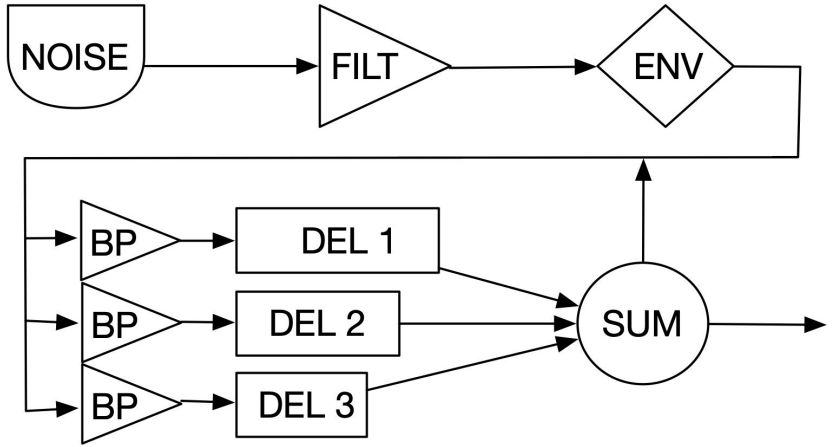

```
// --- audio ---
/* The audioUpdate function is required in all M16 programs. */
int32_t delaySum;

void audioUpdate() {
  int32_t tone = noise.next(); // generate noise exciter
  int32_t filteredTone = noiseFilter.nextBPF(tone); // bandpass filter the noise
  int32_t ampEnv = (filteredTone * env.next()) >> 16; // shape the noise into an impulse
  int32_t feed = clip16(ampEnv + (delaySum >> 6)); // add some cross mod between delays
  delaySum = clip16(delays[0].next(feed) + delays[1].next(feed) + delays[2].next(feed)); // sum three delay lines
  int32_t out = outputGain.next((ampEnv >> 1) + delaySum); // mix the exciter and delays
  audioBlockWrite(out, out); // send to DAC, same signal to L & R
}
```

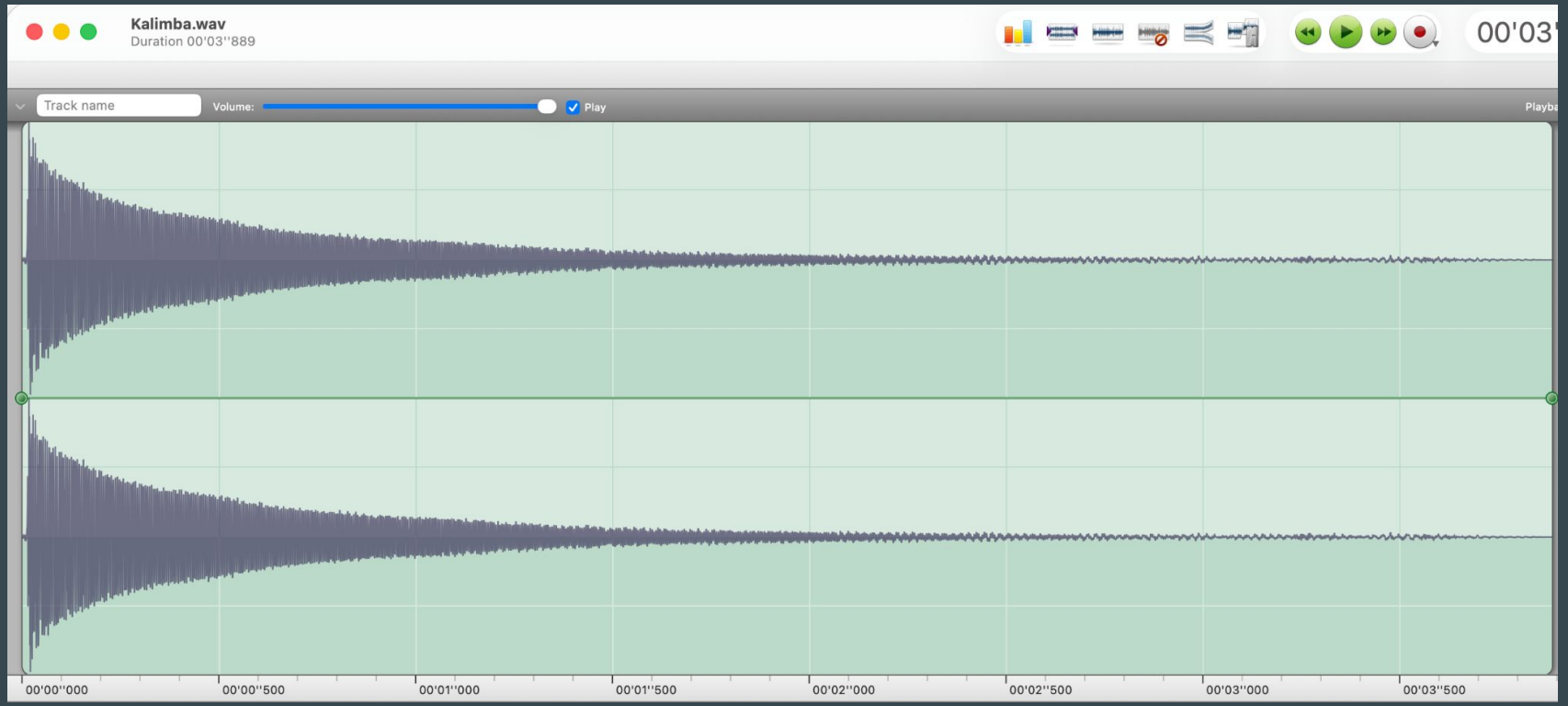


Kalimba

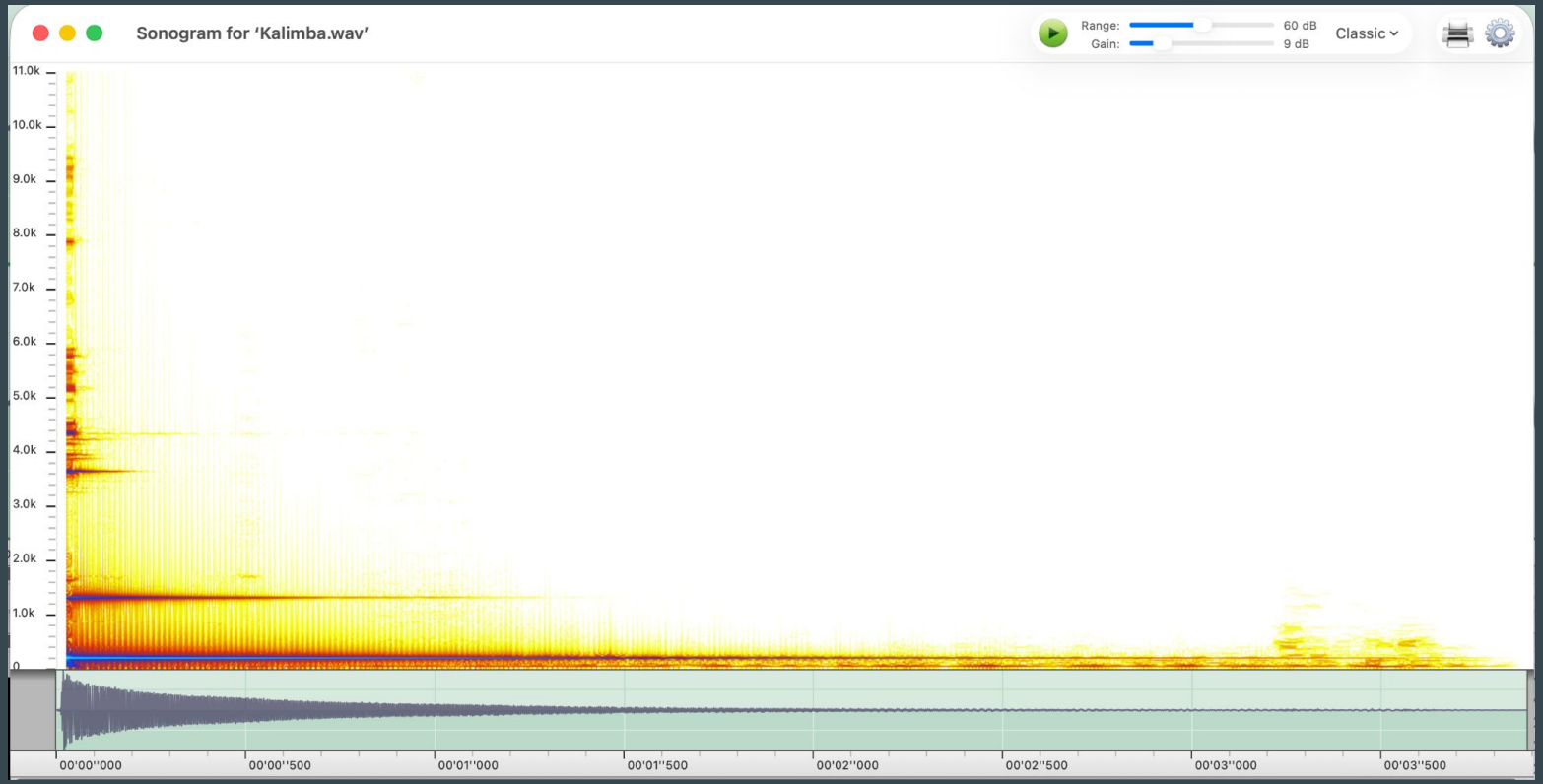
Banded Waveguide



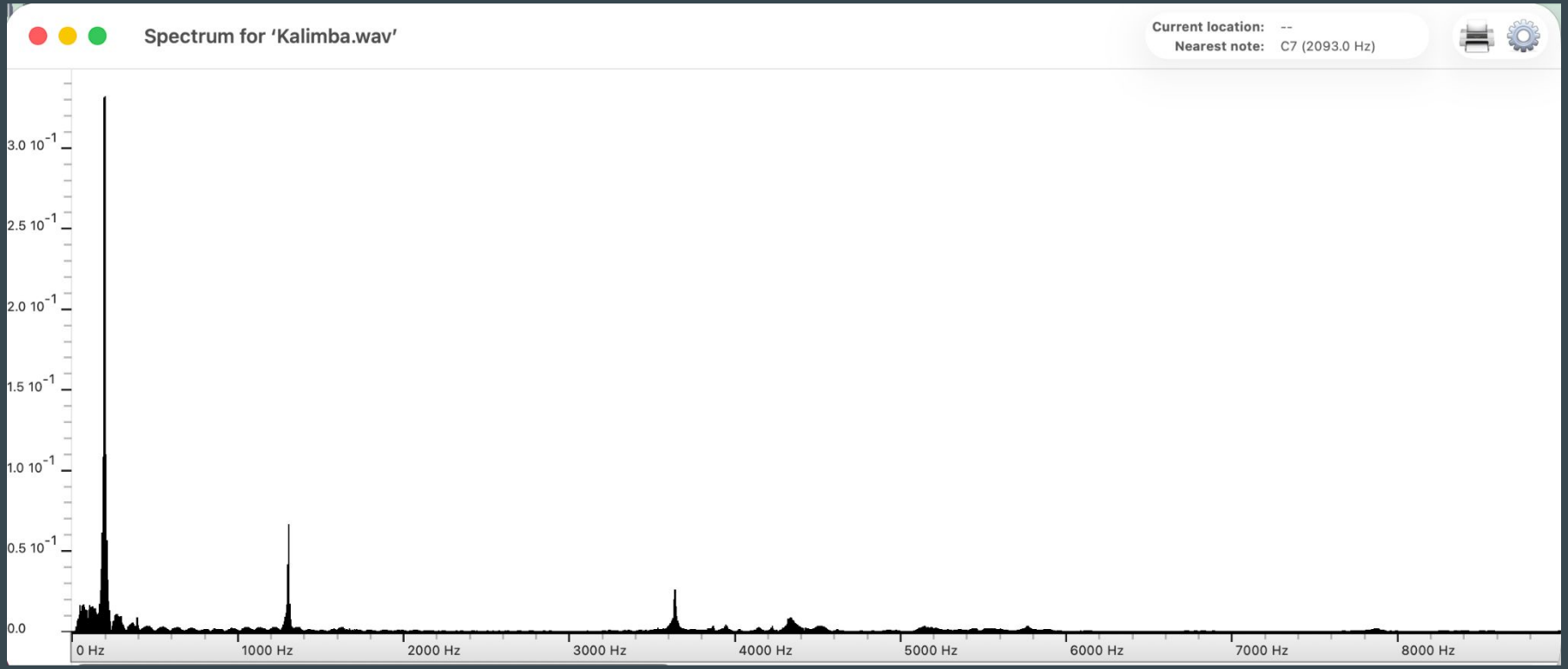
Kalimba Waveform



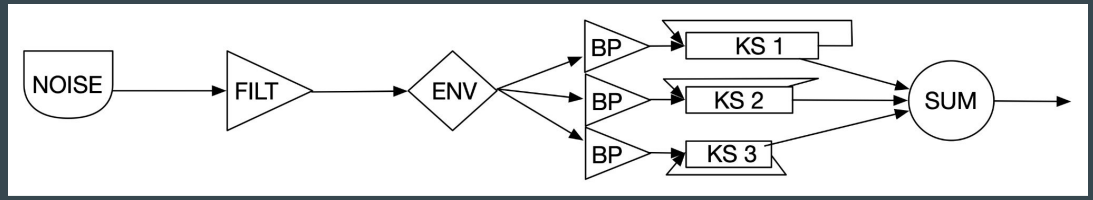
Kalimba Sonogram



Kalimba Spectrum

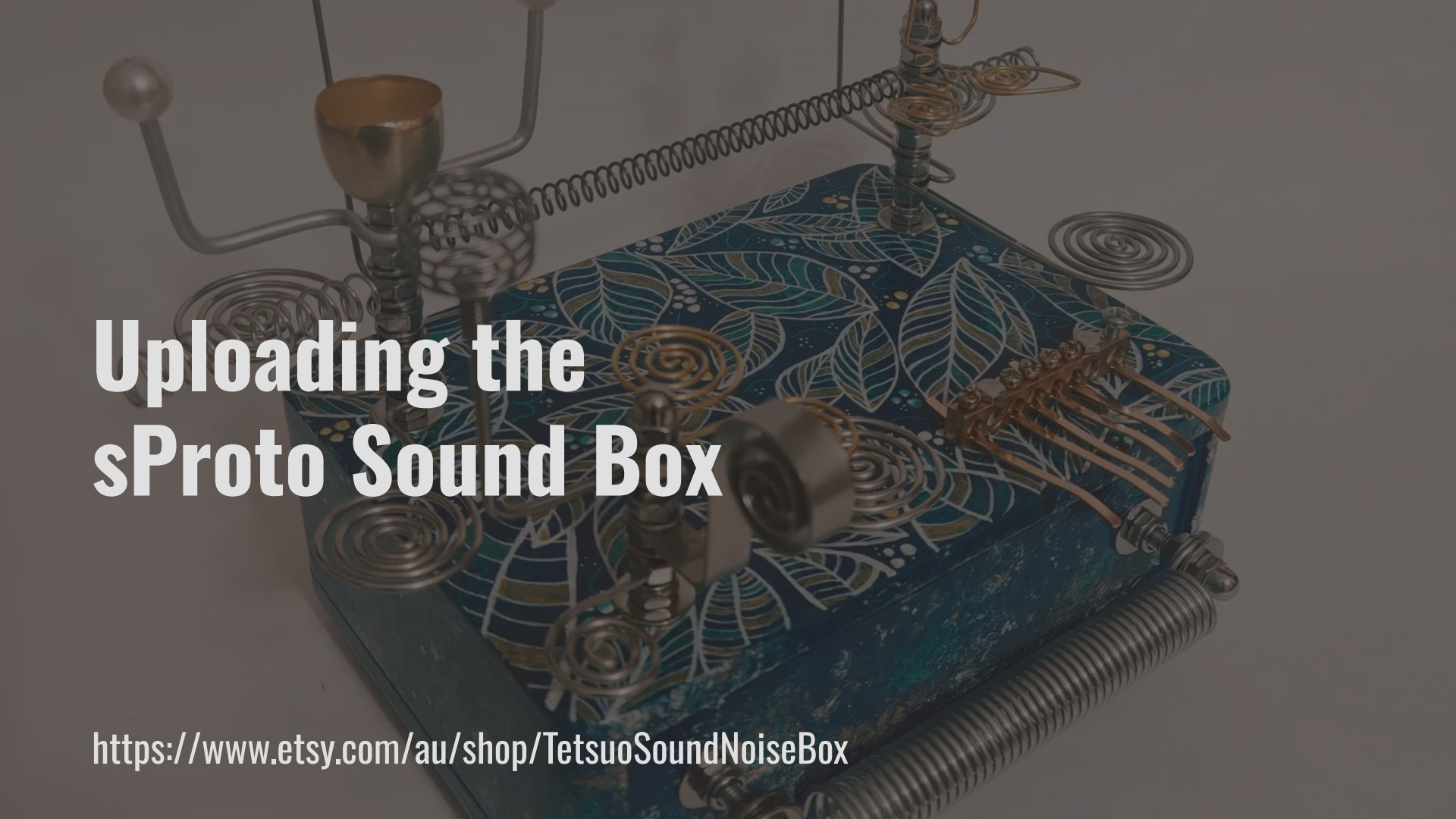


Kalimba synthesis function



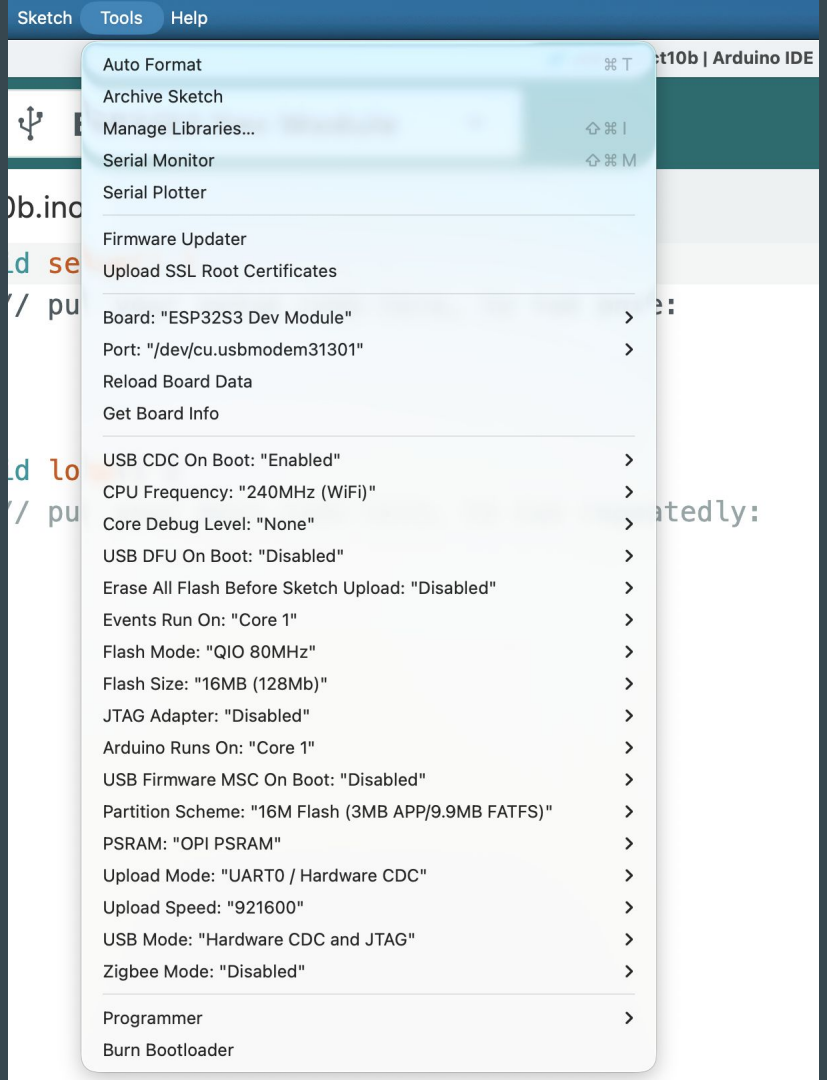

```
/* The audioUpdate function is required in all M16 programs. */
void audioUpdate() {
  int32_t mix = 0; // reused varible for the combined output of all polyphonic voices
  // polyphonic voices are distributed across the 2 cores of the ESP32
  for (int voice = audioPartitionOffset(); voice < voiceCount; voice += audioPartitionStride()) { // for each voice
    int32_t source = noise[voice].nextUnlocked(); // generate noise
    int32_t strike = ((int64_t)source * envelopes[voice].next()) >> 16; // shape noise into an exciter impulse
    int32_t dampenedStrike = exciterFilters[voice].nextLPFUnlocked(strike); // lowpass filter the noise
    // pass exciter through three bandpass filters, then to three tuned Karplus Strong feedback delay lines
    int32_t mode1 = modes[voice][0].pluck(modeFilters[voice][0].nextBPFUnlocked(clip16(dampenedStrike)), feedback1);
    int32_t mode2 = modes[voice][1].pluck(modeFilters[voice][1].nextBPFUnlocked(clip16(dampenedStrike)), feedback2);
    int32_t mode3 = modes[voice][2].pluck(modeFilters[voice][2].nextBPFUnlocked(clip16(dampenedStrike)), feedback3);
    // mix the components together, gain compensating to prevent clipping
    int32_t voiceMix = clip16(((dampenedStrike >> 1) + mode1 + (mode2 >> 1) + (mode3 >> 1)) >> 1);
    mix = clip16(mix + voiceMix);
  }

  audioBlockWrite(mix, mix); // send to the DAC, same mono signal to each L & R channel.
}
```



Uploading the sProto Sound Box

https://www.etsy.com/au/shop/TetsuoSoundNoiseBox



Arduino Settings

The sProto Sound Box sketch needs as much memory as it can get, so ensure these board settings are set in the Tools menu.

Flash Size: 16 MB

Partition Scheme: 16 M Flash

PSRAM: “OPI PSRAM”

sProto Sound Box

Four independent instruments on one sProto board:

- Pad 1 / Dial 1: Tin Whistle, Trigger / Pitch
- Pad 2 / Dial 2: Kalimba, Trigger / Pitch
- Pad 3 / Dial 3: Spring, Trigger / Multiple Triggers
- Pad 4: Glockenspiel, Trigger

Hold the sProto button to adjust effects:

- Dial 1: Delay Time
- Dial 2: Delay Level
- Dial 3: Reverb Level

Coding a Digital Sound Box

On the ESP32 with the sProto PCB & Arduino IDE